Computer Science (joint degree)

Vrije Universiteit Amsterdam - Faculty of Science - Computer Science (joint degree) - 2025-2026

Computer Science studies the technology that has become ubiquitous in our global, connected society. Globally distributed information processing services have taken center stage, with the Internet connecting a wide variety of information processing devices, ranging from mobile phones to data centers operated by leading global companies. The technical side of Computer Science deals with computer operations, like system software, computer networks, data engineering, and programming environments. The theoretical foundations cover the limits of what can be computed, computational efficiency, correctness, and intricacies of concurrent execution. Software engineering studies the construction and maintenance of large and often mission-critical software systems that need to be maintained by large teams of people. Because of its prominent role in everyday life, non-functional aspects of information technology are gaining importance in Computer Science, most notably the energy efficiency of data centers, and the security of computer systems.

The Master programme Computer Science is structured in four specialisations, from which a student chooses one according to personal preference.

- Big Data Engineering (BDE)
- Computer Systems and Infrastructure (CSI)
- Foundations of Computing and Concurrency (FCC)
- Software Engineering and Green IT (SEG)

The Master programme Computer Science is a joint degree programme of the Vrije Universiteit Amsterdam and the University of Amsterdam. Courses are taught at locations of both universities. Courses run by the UvA. can be found in the UvA studyguide: Programme | UvA Studiegids 2025 - 2026.

Info

Level	Master
Language	English
Duration	2 years
Type	Full Time
Credits	120 EC
Faculty	Faculty of Science

Joint degree
Teaching and examination regulation 2025-2026
More information about this programme

Computer Science specialisation Big Data Engineering 5
Computer Science specialisation Big Data Engineering constrained choice 5
CS constrained choice FCC core 5
CS constrained choice BDE core 5
CS constrained choice CSI core 6
Computer Science JD Mathematics constrained choice year 1 and year 2 6
Computer Science JD Societal Perspectives on Computer Science constrained choice 6
Computer Sciences JD Security constrained choice 6
CS constrained choice SEG core 6
Computer Science pre-approved elective courses 7
Computer Science specialisation Computer Systems and Infrastructure 8
Computer Science specialisation Computer Systems and Infrastructure constrained choice 8
CS constrained choice FCC core 9
CS constrained choice BDE core 9
CS constrained choice CSI core 9
Computer Science JD Mathematics constrained choice year 1 and year 2 9
Computer Science JD Societal Perspectives on Computer Science constrained choice 9
Computer Sciences JD Security constrained choice 10
CS constrained choice SEG core 10
Computer Science pre-approved elective courses 10
Computer Science specialisation Foundations of Computing and Concurrency 11
Computer Science specialisation Foundations of Computing & Concurrency year 1 constrained choice 12
CS constrained choice FCC core 12
CS constrained choice BDE core 12
CS constrained choice CSI core 12
Computer Science JD Mathematics constrained choice year 1 and year 2 13
Computer Science JD Societal Perspectives on Computer Science constrained choice 13
Computer Sciences JD Security constrained choice 13
CS constrained choice SEG core 13
Computer Science pre-approved elective courses 13
Computer Science specialisation Software Engineering and Green IT 15
Computer Science pre-approved elective courses 15
Computer Science specialisation Software Engineering & Green IT constrained choice 16
CS constrained choice FCC core 17
CS constrained choice BDE core 17
CS constrained choice CSI core 17
Computer Science JD Mathematics constrained choice year 1 and year 2 17
Computer Science JD Societal Perspectives on Computer Science constrained choice 17
Computer Sciences JD Security constrained choice 18
CS constrained choice SEG core 18
Courses 18
Accelerator-Centric Computing Ecosystems 18
Advanced Algorithms 20
Advanced Graph Algorithms 20
Advanced Logic 21
Advanced Operating Systems 22
AI for Security 24
Algorithmic Game Theory 26
Computational Complexity 27
Cryptographic Engineering 27
Data Mining Techniques 28

Data Preparation 29
Data Protection Technologies 30
Digital Architecture 30
Digital Business Ecosystems and Platforms 32
Digital Twin Engineering 34
Digitalization and Sustainability 34
Distributed Algorithms 36
Distributed Systems 37
Dynamic Programming and Reinforcement Learning 39
E-Commerce Law 40
Energy-efficient Edge Computing 41
Entrepreneurship in Analytics and AI 42
Evolutionary Computing 43
Experimental Design and Data Analysis 44
Foundations of Cryptography 45
Functional Programming 46
Fundamentals of Adaptive Software 47
Green Lab 48
High Performance Computing and Big Data 50
History of Digital Cultures 50
ICT for the Global South 51
Inclusive ICT 53
Information Theoretic Learning 54
Introduction to Computational Science 54
Knowledge Organization 55
Large Research Project Computer Science 56
Literature Study 57
Logical Verification 59
Machine Learning for the Quantified Self 60
Master Project Computer Science 61
Multi-core Processor Systems 63
Network Security 63
Parallel Programming Practical 64
Performance of Networked Systems 66
Programming Large-Scale Parallel Systems 67
Research Methods in Computer Science 68
Research Project Computer Science 69
Secure and Trustworthy Machine Learning 70
Secure Computation 72
Security Experiments and Measurements 73
Service Oriented Design 76
Software Language Engineering 78
Mandatory Literature: 80
Recommended Literature: 80
Software Security 80
Software Testing 81
Systems Seminar 83
Term Rewriting Systems 84
The Social Web 85
Verification for Security 87
Web Services and Cloud-Based Systems 88

Computer Science specialisation Big Data Engineering

Description

This specialisation concerns the technology that underlies global data-processing infrastructures. Students acquire deep technical knowledge on data science and on connecting the whole range, from small company computer clusters to data centers run by world-leading IT giants. As such, graduates are experts in computer systems for processing, analyzing, and managing large quantities of data.

Specialisation Coordinator:
Dr. Victoria Degeler
v.o.degeler@uva.nl

Courses

Course Name	Period	Credits	Code
Master Project Computer Science	Ac. Year (September)	30EC	XM_0011
Research Methods in Computer Science	P6	6EC	XM_0175

Programme Sections

- Computer Science specialisation Big Data Engineering constrained choice
- Computer Science pre-approved elective courses

Computer Science specialisation Big Data Engineering constrained choice

Description

The curriculum of the Master Computer Science consists of several constrained choice categories, together ensuring the breadth of the study plan.

Programme Sections

- CS constrained choice FCC core
- CS constrained choice BDE core
- CS constrained choice CSI core
- Computer Science JD Mathematics constrained choice year 1 and year 2
- Computer Science JD Societal Perspectives on Computer Science constrained choice
- Computer Sciences JD Security constrained choice
- CS constrained choice SEG core

CS constrained choice FCC core

Courses

Course Name	Period	Credits	Code
Logical Verification	P1	6EC	XM_0167
Advanced Algorithms	P2	6EC	XMU_0060
Foundations of Cryptography	P2	6EC	XM_0174
Information Theoretic Learning	P2	6EC	XMU_0064
Advanced Logic	P4	6EC	X_405048
Term Rewriting Systems	P4	6EC	XM_400121
Advanced Graph Algorithms	P5	6EC	XM_0173
Distributed Algorithms	P5	6EC	X_400211

CS constrained choice BDE core

Courses

Course Name	Period	Credits	Code

Course Name	Period	Credits	Code
Digital Twin Engineering	P1	6EC	XMU_0068
Data Preparation	P3	6EC	XMU_0058
Data Protection Technologies	P4	6EC	XMU_0057
Web Services and Cloud-Based Systems	P4	6EC	XMU_418110
Data Mining Techniques	P5	6EC	X_400108

CS constrained choice CSI core

Courses

Course Name	Period	Credits	Code
Advanced Operating Systems	P1	6EC	XM_40014
Programming Large-Scale Parallel Systems	P1	6EC	XM_40017
Distributed Systems	P2	6EC	X_400130
Parallel Programming Practical	P3	6EC	X_400162
Multi-core Processor Systems	P4	6EC	XMU_0055
Accelerator-Centric Computing Ecosystems	P5	6EC	XM_0171
Energy-efficient Edge Computing	P5	6EC	XMU_0056

Computer Science JD Mathematics constrained choice year 1 and year 2

Courses

Course Name	Period	Credits	Code
Dynamic Programming and Reinforcement Learning	P2	6EC	XM_0093
Performance of Networked Systems	P2	6EC	X_405105
Experimental Design and Data Analysis	P4	6EC	X_405078

Computer Science JD Societal Perspectives on Computer Science constrained choice

Courses

Course Name	Period	Credits	Code
E-Commerce Law	P3	6EC	R_E.commerc
History of Digital Cultures	P3	6EC	XM_0134
Entrepreneurship in Analytics and AI	P4	6EC	XM_0090
ICT for the Global South	P5	6EC	XM_0142
Inclusive ICT	P6	6EC	XM_0177

Computer Sciences JD Security constrained choice

Courses

Course Name	Period	Credits	Code
Software Security	P1	6EC	X_400127
Verification for Security	P1	6EC	XM_0099
Network Security	P2	6EC	XM_0100
Secure and Trustworthy Machine Learning	P3	6EC	XM_0179
Security Experiments and Measurements	P3	6EC	XM_0098
Cryptographic Engineering	P4	6EC	XMU_0047
Secure Computation	P5	6EC	XMU_0065
AI for Security	P6	6EC	XM_0178

CS constrained choice SEG core

Courses

Course Name	Period	Credits	Code

Course Name	Period	Credits	Code
Digitalization and Sustainability	P1	6EC	XM_0089
Green Lab	P1	6EC	X_418158
Service Oriented Design	P1	6EC	X_405061
Digital Architecture	P2	6EC	XM_0127
Fundamentals of Adaptive Software	P2	6EC	XM_0128
Software Language Engineering	P4	6EC	XM_0172
Software Testing	P5	6EC	X_400439

Computer Science pre-approved elective courses

Courses

Course Name	Period	Credits	Code
Large Research Project Computer Science	Ac. Year (September)	12EC	XM_0130
Literature Study	Ac. Year (September)	6EC	XM_0131
Research Project Computer Science	Ac. Year (September)	6EC	XM_0129
Advanced Operating Systems	P1	6EC	XM_40014
Algorithmic Game Theory	P1	6EC	XMU_0063
Digital Twin Engineering	P1	6EC	XMU_0068
Digitalization and Sustainability	P1	6EC	XM_0089
Evolutionary Computing	P1	6EC	X_400111
Green Lab	P1	6EC	X_418158
Introduction to Computational Science	P1	6EC	XMU_418111
Knowledge Organization	P1	6EC	X_405065
Logical Verification	P1	6EC	XM_0167
Programming Large-Scale Parallel Systems	P1	6EC	XM_40017
Service Oriented Design	P1	6EC	X_405061
Software Security	P1	6EC	X_400127
Verification for Security	P1	6EC	XM_0099
Advanced Algorithms	P2	6EC	XMU_0060
Digital Architecture	P2	6EC	XM_0127
Distributed Systems	P2	6EC	X_400130
Dynamic Programming and Reinforcement Learning	P2	6EC	XM_0093
Fundamentals of Adaptive Software	P2	6EC	XM_0128
Information Theoretic Learning	P2	6EC	XMU_0064
Network Security	P2	6EC	XM_0100
Performance of Networked Systems	P2	6EC	X_405105
The Social Web	P2	6EC	X_405086
Data Preparation	P3	6EC	XMU_0058
E-Commerce Law	P3	6EC	R_E.commerc
High Performance Computing and Big Data	P3	6EC	XMU_40013
History of Digital Cultures	P3	6EC	XM_0134
Parallel Programming Practical	P3	6EC	X_400162
Secure and Trustworthy Machine Learning	P3	6EC	XM_0179
Security Experiments and Measurements	P3	6EC	XM_0098
Advanced Logic	P4	6EC	X_405048
Cryptographic Engineering	P4	6EC	XMU_0047
Data Protection Technologies	P4	6EC	XMU_0057
Digital Business Ecosystems and Platforms	P4	6EC	XM_0176
Entrepreneurship in Analytics and AI	P4	6EC	XM_0090
Experimental Design and Data Analysis	P4	6EC	X_405078
Functional Programming	P4	6EC	XMU_0062
Multi-core Processor Systems	P4	6EC	XMU_0055
Software Language Engineering	P4	6EC	XM_0172
Systems Seminar	P4	6EC	XM_0122

Course Name	Period	Credits	Code
Term Rewriting Systems	P4	6EC	XM_400121
Web Services and Cloud-Based Systems	P4	6EC	XMU_418110
Accelerator-Centric Computing Ecosystems	P5	6EC	XM_0171
Advanced Graph Algorithms	P5	6EC	XM_0173
Computational Complexity	P5	6EC	XMU_0061
Data Mining Techniques	P5	6EC	X_400108
Distributed Algorithms	P5	6EC	X_400211
Energy-efficient Edge Computing	P5	6EC	XMU_0056
ICT for the Global South	P5	6EC	XM_0142
Secure Computation	P5	6EC	XMU_0065
Software Testing	P5	6EC	X_400439
AI for Security	P6	6EC	XM_0178
Inclusive ICT	P6	6EC	XM_0177
Machine Learning for the Quantified Self	P6	6EC	XM_40012

Computer Science specialisation Computer Systems and Infrastructure

Description

This specialisation focuses on computer system and network infrastructures, which have become the core components of all distributed data and information processing systems that our modern society depends on. Many computer systems are even not recognizable as such, because they are embedded in larger devices such as cars, airplanes, medical equipment, smart buildings, robotics, etc.

The realization of modern computer systems is greatly complicated by their increasing complexity: these computer systems integrate increasingly more processors, often heterogeneous, sometimes in a distributed context, and possibly also resulting in complex systems-of-systems (such as can be found, e.g., in the Edge-to-Cloud computing continuum and in the high-performance memory-storage system). Meanwhile, computer networks interconnecting these systems are evolving with significantly increased customizability in addition to higher speeds. The energy consumption, sustainability, dependability, and security and privacy of these computer systems and networks also require increasing attention.

Specialisation coordinator:
Dr. Tiziano De Matteis
t.de.matteis@vu.nl

Courses

Course Name	Period	Credits	Code
Master Project Computer Science	Ac. Year (September)	30EC	XM_0011
Research Methods in Computer Science	P6	6EC	XM_0175

Programme Sections

- Computer Science specialisation Computer Systems and Infrastructure constrained choice
- Computer Science pre-approved elective courses

Computer Science specialisation Computer Systems and Infrastructure constrained choice

Description

The curriculum of the Master Computer Science consists of several constrained choice categories, together ensuring the breadth of the study plan.

Programme Sections

- CS constrained choice FCC core
- CS constrained choice BDE core
- CS constrained choice CSI core

- Computer Science JD Mathematics constrained choice year 1 and year 2
- Computer Science JD Societal Perspectives on Computer Science constrained choice
- Computer Sciences JD Security constrained choice
- CS constrained choice SEG core

CS constrained choice FCC core

Courses

Course Name	Period	Credits	Code
Logical Verification	P1	6EC	XM_0167
Advanced Algorithms	P2	6EC	XMU_0060
Foundations of Cryptography	P2	6EC	XM_0174
Information Theoretic Learning	P2	6EC	XMU_0064
Advanced Logic	P4	6EC	X_405048
Term Rewriting Systems	P4	6EC	XM_400121
Advanced Graph Algorithms	P5	6EC	XM_0173
Distributed Algorithms	P5	6EC	X_400211

CS constrained choice BDE core

Courses

Course Name	Period	Credits	Code
Digital Twin Engineering	P1	6EC	XMU_0068
Data Preparation	P3	6EC	XMU_0058
Data Protection Technologies	P4	6EC	XMU_0057
Web Services and Cloud-Based Systems	P4	6EC	XMU_418110
Data Mining Techniques	P5	6EC	X_400108

CS constrained choice CSI core

Courses

Course Name	Period	Credits	Code
Advanced Operating Systems	P1	6EC	XM_40014
Programming Large-Scale Parallel Systems	P1	6EC	XM_40017
Distributed Systems	P2	6EC	X_400130
Parallel Programming Practical	P3	6EC	X_400162
Multi-core Processor Systems	P4	6EC	XMU_0055
Accelerator-Centric Computing Ecosystems	P5	6EC	XM_0171
Energy-efficient Edge Computing	P5	6EC	XMU_0056

Computer Science JD Mathematics constrained choice year 1 and year 2

Courses

Course Name	Period	Credits	Code
Dynamic Programming and Reinforcement Learning	P2	6EC	XM_0093
Performance of Networked Systems	P2	6EC	X_405105
Experimental Design and Data Analysis	P4	6EC	X_405078

Computer Science JD Societal Perspectives on Computer Science constrained choice

Courses

Course Name	Period	Credits	Code
E-Commerce Law	P3	6EC	R_E.commerc
History of Digital Cultures	P3	6EC	XM_0134
Entrepreneurship in Analytics and AI	P4	6EC	XM_0090

Course Name	Period	Credits	Code
ICT for the Global South	P5	6EC	XM_0142
Inclusive ICT	P6	6EC	XM_0177

Computer Sciences JD Security constrained choice

Courses

Course Name	Period	Credits	Code
Software Security	P1	6EC	X_400127
Verification for Security	P1	6EC	XM_0099
Network Security	P2	6EC	XM_0100
Secure and Trustworthy Machine Learning	P3	6EC	XM_0179
Security Experiments and Measurements	P3	6EC	XM_0098
Cryptographic Engineering	P4	6EC	XMU_0047
Secure Computation	P5	6EC	XMU_0065
AI for Security	P6	6EC	XM_0178

CS constrained choice SEG core

Courses

Course Name	Period	Credits	Code
Digitalization and Sustainability	P1	6EC	XM_0089
Green Lab	P1	6EC	X_418158
Service Oriented Design	P1	6EC	X_405061
Digital Architecture	P2	6EC	XM_0127
Fundamentals of Adaptive Software	P2	6EC	XM_0128
Software Language Engineering	P4	6EC	XM_0172
Software Testing	P5	6EC	X_400439

Computer Science pre-approved elective courses

Courses

Course Name	Period	Credits	Code
Large Research Project Computer Science	Ac. Year (September)	12EC	XM_0130
Literature Study	Ac. Year (September)	6EC	XM_0131
Research Project Computer Science	Ac. Year (September)	6EC	XM_0129
Advanced Operating Systems	P1	6EC	XM_40014
Algorithmic Game Theory	P1	6EC	XMU_0063
Digital Twin Engineering	P1	6EC	XMU_0068
Digitalization and Sustainability	P1	6EC	XM_0089
Evolutionary Computing	P1	6EC	X_400111
Green Lab	P1	6EC	X_418158
Introduction to Computational Science	P1	6EC	XMU_418111
Knowledge Organization	P1	6EC	X_405065
Logical Verification	P1	6EC	XM_0167
Programming Large-Scale Parallel Systems	P1	6EC	XM_40017
Service Oriented Design	P1	6EC	X_405061
Software Security	P1	6EC	X_400127
Verification for Security	P1	6EC	XM_0099
Advanced Algorithms	P2	6EC	XMU_0060
Digital Architecture	P2	6EC	XM_0127
Distributed Systems	P2	6EC	X_400130
Dynamic Programming and Reinforcement Learning	P2	6EC	XM_0093
Fundamentals of Adaptive Software	P2	6EC	XM_0128

Course Name	Period	Credits	Code
Information Theoretic Learning	P2	6EC	XMU_0064
Network Security	P2	6EC	XM_0100
Performance of Networked Systems	P2	6EC	X_405105
The Social Web	P2	6EC	X_405086
Data Preparation	P3	6EC	XMU_0058
E-Commerce Law	P3	6EC	R_E.commerc
High Performance Computing and Big Data	P3	6EC	XMU_40013
History of Digital Cultures	P3	6EC	XM_0134
Parallel Programming Practical	P3	6EC	X_400162
Secure and Trustworthy Machine Learning	P3	6EC	XM_0179
Security Experiments and Measurements	P3	6EC	XM_0098
Advanced Logic	P4	6EC	X_405048
Cryptographic Engineering	P4	6EC	XMU_0047
Data Protection Technologies	P4	6EC	XMU_0057
Digital Business Ecosystems and Platforms	P4	6EC	XM_0176
Entrepreneurship in Analytics and AI	P4	6EC	XM_0090
Experimental Design and Data Analysis	P4	6EC	X_405078
Functional Programming	P4	6EC	XMU_0062
Multi-core Processor Systems	P4	6EC	XMU_0055
Software Language Engineering	P4	6EC	XM_0172
Systems Seminar	P4	6EC	XM_0122
Term Rewriting Systems	P4	6EC	XM_400121
Web Services and Cloud-Based Systems	P4	6EC	XMU_418110
Accelerator-Centric Computing Ecosystems	P5	6EC	XM_0171
Advanced Graph Algorithms	P5	6EC	XM_0173
Computational Complexity	P5	6EC	XMU_0061
Data Mining Techniques	P5	6EC	X_400108
Distributed Algorithms	P5	6EC	X_400211
Energy-efficient Edge Computing	P5	6EC	XMU_0056
ICT for the Global South	P5	6EC	XM_0142
Secure Computation	P5	6EC	XMU_0065
Software Testing	P5	6EC	X_400439
AI for Security	P6	6EC	XM_0178
Inclusive ICT	P6	6EC	XM_0177
Machine Learning for the Quantified Self	P6	6EC	XM_40012

Computer Science specialisation Foundations of Computing and Concurrency

Description

This specialisation focuses on formal methods in computing, concurrency and the design and verification of software systems. Some theoretical disciplines that play a central role are term rewriting, process algebra, distributed algorithms and type theory. Foundational disciplines include logic, recursion theory and complexity. These topics have a range of applications, of which we mention a few. Tools developed from process algebra are used in protocol validation. Term rewriting lies at the basis of functional programming and the analysis of infinitary processes. Distributed algorithms are of central importance for the efficient use of concurrent systems. Logic and type theory form the basis of proof checking, used in software verification.

Specialisation Coordinator:
Dr. Femke van Raamsdonk
f.van.raamsdonk@vu.nl

Courses

Course Name	Period	Credits	Code
Master Project Computer Science	Ac. Year (September)	30EC	XM_0011

Course Name	Period	Credits	Code
Research Methods in Computer Science	P6	6EC	XM_0175

Programme Sections

- Computer Science specialisation Foundations of Computing & Concurrency year 1 constrained choice
- Computer Science pre-approved elective courses

Computer Science specialisation Foundations of Computing & Concurrency year 1 constrained choice

Description

The curriculum of the Master Computer Science consists of several constrained choice categories, together ensuring the breadth of the study plan.

Programme Sections

- CS constrained choice FCC core
- CS constrained choice BDE core
- CS constrained choice CSI core
- Computer Science JD Mathematics constrained choice year 1 and year 2
- Computer Science JD Societal Perspectives on Computer Science constrained choice
- Computer Sciences JD Security constrained choice
- CS constrained choice SEG core

CS constrained choice FCC core

Courses

Course Name	Period	Credits	Code
Logical Verification	P1	6EC	XM_0167
Advanced Algorithms	P2	6EC	XMU_0060
Foundations of Cryptography	P2	6EC	XM_0174
Information Theoretic Learning	P2	6EC	XMU_0064
Advanced Logic	P4	6EC	X_405048
Term Rewriting Systems	P4	6EC	XM_400121
Advanced Graph Algorithms	P5	6EC	XM_0173
Distributed Algorithms	P5	6EC	X_400211

CS constrained choice BDE core

Courses

Course Name	Period	Credits	Code
Digital Twin Engineering	P1	6EC	XMU_0068
Data Preparation	P3	6EC	XMU_0058
Data Protection Technologies	P4	6EC	XMU_0057
Web Services and Cloud-Based Systems	P4	6EC	XMU_418110
Data Mining Techniques	P5	6EC	X_400108

CS constrained choice CSI core

Courses

Course Name	Period	Credits	Code
Advanced Operating Systems	P1	6EC	XM_40014
Programming Large-Scale Parallel Systems	P1	6EC	XM_40017
Distributed Systems	P2	6EC	X_400130
Parallel Programming Practical	P3	6EC	X_400162
Multi-core Processor Systems	P4	6EC	XMU_0055
Accelerator-Centric Computing Ecosystems	P5	6EC	XM_0171

Course Name	Period	Credits	Code
Energy-efficient Edge Computing	P5	6EC	XMU_0056

Computer Science JD Mathematics constrained choice year 1 and year 2

Courses

Course Name	Period	Credits	Code
Dynamic Programming and Reinforcement Learning	P2	6EC	XM_0093
Performance of Networked Systems	P2	6EC	X_405105
Experimental Design and Data Analysis	P4	6EC	X_405078

Computer Science JD Societal Perspectives on Computer Science constrained choice

Courses

Course Name	Period	Credits	Code
E-Commerce Law	P3	6EC	R_E.commerc
History of Digital Cultures	P3	6EC	XM_0134
Entrepreneurship in Analytics and AI	P4	6EC	XM_0090
ICT for the Global South	P5	6EC	XM_0142
Inclusive ICT	P6	6EC	XM_0177

Computer Sciences JD Security constrained choice

Courses

Course Name	Period	Credits	Code
Software Security	P1	6EC	X_400127
Verification for Security	P1	6EC	XM_0099
Network Security	P2	6EC	XM_0100
Secure and Trustworthy Machine Learning	P3	6EC	XM_0179
Security Experiments and Measurements	P3	6EC	XM_0098
Cryptographic Engineering	P4	6EC	XMU_0047
Secure Computation	P5	6EC	XMU_0065
AI for Security	P6	6EC	XM_0178

CS constrained choice SEG core

Courses

Course Name	Period	Credits	Code
Digitalization and Sustainability	P1	6EC	XM_0089
Green Lab	P1	6EC	X_418158
Service Oriented Design	P1	6EC	X_405061
Digital Architecture	P2	6EC	XM_0127
Fundamentals of Adaptive Software	P2	6EC	XM_0128
Software Language Engineering	P4	6EC	XM_0172
Software Testing	P5	6EC	X_400439

Computer Science pre-approved elective courses

Courses

Course Name	Period	Credits	Code
Large Research Project Computer Science	Ac. Year (September)	12EC	XM_0130

Course Name	Period	Credits	Code
Literature Study	Ac. Year (September)	6EC	XM_0131
Research Project Computer Science	Ac. Year (September)	6EC	XM_0129
Advanced Operating Systems	P1	6EC	XM_40014
Algorithmic Game Theory	P1	6EC	XMU_0063
Digital Twin Engineering	P1	6EC	XMU_0068
Digitalization and Sustainability	P1	6EC	XM_0089
Evolutionary Computing	P1	6EC	X_400111
Green Lab	P1	6EC	X_418158
Introduction to Computational Science	P1	6EC	XMU_418111
Knowledge Organization	P1	6EC	X_405065
Logical Verification	P1	6EC	XM_0167
Programming Large-Scale Parallel Systems	P1	6EC	XM_40017
Service Oriented Design	P1	6EC	X_405061
Software Security	P1	6EC	X_400127
Verification for Security	P1	6EC	XM_0099
Advanced Algorithms	P2	6EC	XMU_0060
Digital Architecture	P2	6EC	XM_0127
Distributed Systems	P2	6EC	X_400130
Dynamic Programming and Reinforcement Learning	P2	6EC	XM_0093
Fundamentals of Adaptive Software	P2	6EC	XM_0128
Information Theoretic Learning	P2	6EC	XMU_0064
Network Security	P2	6EC	XM_0100
Performance of Networked Systems	P2	6EC	X_405105
The Social Web	P2	6EC	X_405086
Data Preparation	P3	6EC	XMU_0058
E-Commerce Law	P3	6EC	R_E.commerc
High Performance Computing and Big Data	P3	6EC	XMU_40013
History of Digital Cultures	P3	6EC	XM_0134
Parallel Programming Practical	P3	6EC	X_400162
Secure and Trustworthy Machine Learning	P3	6EC	XM_0179
Security Experiments and Measurements	P3	6EC	XM_0098
Advanced Logic	P4	6EC	X_405048
Cryptographic Engineering	P4	6EC	XMU_0047
Data Protection Technologies	P4	6EC	XMU_0057
Digital Business Ecosystems and Platforms	P4	6EC	XM_0176
Entrepreneurship in Analytics and AI	P4	6EC	XM_0090
Experimental Design and Data Analysis	P4	6EC	X_405078
Functional Programming	P4	6EC	XMU_0062
Multi-core Processor Systems	P4	6EC	XMU_0055
Software Language Engineering	P4	6EC	XM_0172
Systems Seminar	P4	6EC	XM_0122
Term Rewriting Systems	P4	6EC	XM_400121
Web Services and Cloud-Based Systems	P4	6EC	XMU_418110
Accelerator-Centric Computing Ecosystems	P5	6EC	XM_0171
Advanced Graph Algorithms	P5	6EC	XM_0173
Computational Complexity	P5	6EC	XMU_0061
Data Mining Techniques	P5	6EC	X_400108
Distributed Algorithms	P5	6EC	X_400211
Energy-efficient Edge Computing	P5	6EC	XMU_0056
ICT for the Global South	P5	6EC	XM_0142
Secure Computation	P5	6EC	XMU_0065
Software Testing	P5	6EC	X_400439
AI for Security	P6	6EC	XM_0178
Inclusive ICT	P6	6EC	XM_0177
Machine Learning for the Quantified Self	P6	6EC	XM_40012

Computer Science specialisation Software Engineering and Green IT

Description

The specialisation in Software Engineering and Green IT offers a unique combination: 1)**Software engineering** applies a systematic and quantifiable approach to the development, execution, and maintenance of complex software and 2) **Green IT** is the study and practice of environmentally sustainable computing. The combination of Software Engineering and Green IT in one track provides the students with the instruments necessary to gain a holistic understanding of large-scale and complex software systems, to manage their evolution, assess their quality and environmental impact, quantify their value and sustainability potential, and organize their development in different local- and distributed contexts.

Students graduating in the Software Engineering and Green IT specialisation are experts of:

- architecture design of software-intensive systems
- the role of software for sustainability (including energy efficiency, socio-technical, ecological, and economic impact)
- software engineering techniques for critical analysis and decision-making
- benefits and challenges of developing and maintaining sustainable software
- the pervasive role of software-intensive systems in the digital society
- data-driven measurement and assessment of software quality

Specialisation Coordinators:

- Dr. Ivano Malavolta (i.malavolta@vu.nl)
- Abhishek Subramanian (a.s.iyer@student.vu.nl)

Courses

Course Name	Period	Credits	Code
Master Project Computer Science	Ac. Year (September)	30EC	XM_0011
Research Methods in Computer Science	P6	6EC	XM_0175

Programme Sections

- Computer Science pre-approved elective courses
- Computer Science specialisation Software Engineering & Green IT constrained choice

Computer Science pre-approved elective courses

Courses

Course Name	Period	Credits	Code
Large Research Project Computer Science	Ac. Year (September)	12EC	XM_0130
Literature Study	Ac. Year (September)	6EC	XM_0131
Research Project Computer Science	Ac. Year (September)	6EC	XM_0129
Advanced Operating Systems	P1	6EC	XM_40014
Algorithmic Game Theory	P1	6EC	XMU_0063
Digital Twin Engineering	P1	6EC	XMU_0068
Digitalization and Sustainability	P1	6EC	XM_0089
Evolutionary Computing	P1	6EC	X_400111
Green Lab	P1	6EC	X_418158
Introduction to Computational Science	P1	6EC	XMU_418111
Knowledge Organization	P1	6EC	X_405065
Logical Verification	P1	6EC	XM_0167
Programming Large-Scale Parallel Systems	P1	6EC	XM_40017
Service Oriented Design	P1	6EC	X_405061
Software Security	P1	6EC	X_400127

Course Name	Period	Credits	Code
Verification for Security	P1	6EC	XM_0099
Advanced Algorithms	P2	6EC	XMU_0060
Digital Architecture	P2	6EC	XM_0127
Distributed Systems	P2	6EC	X_400130
Dynamic Programming and Reinforcement Learning	P2	6EC	XM_0093
Fundamentals of Adaptive Software	P2	6EC	XM_0128
Information Theoretic Learning	P2	6EC	XMU_0064
Network Security	P2	6EC	XM_0100
Performance of Networked Systems	P2	6EC	X_405105
The Social Web	P2	6EC	X_405086
Data Preparation	P3	6EC	XMU_0058
E-Commerce Law	P3	6EC	R_E.commerc
High Performance Computing and Big Data	P3	6EC	XMU_40013
History of Digital Cultures	P3	6EC	XM_0134
Parallel Programming Practical	P3	6EC	X_400162
Secure and Trustworthy Machine Learning	P3	6EC	XM_0179
Security Experiments and Measurements	P3	6EC	XM_0098
Advanced Logic	P4	6EC	X_405048
Cryptographic Engineering	P4	6EC	XMU_0047
Data Protection Technologies	P4	6EC	XMU_0057
Digital Business Ecosystems and Platforms	P4	6EC	XM_0176
Entrepreneurship in Analytics and AI	P4	6EC	XM_0090
Experimental Design and Data Analysis	P4	6EC	X_405078
Functional Programming	P4	6EC	XMU_0062
Multi-core Processor Systems	P4	6EC	XMU_0055
Software Language Engineering	P4	6EC	XM_0172
Systems Seminar	P4	6EC	XM_0122
Term Rewriting Systems	P4	6EC	XM_400121
Web Services and Cloud-Based Systems	P4	6EC	XMU_418110
Accelerator-Centric Computing Ecosystems	P5	6EC	XM_0171
Advanced Graph Algorithms	P5	6EC	XM_0173
Computational Complexity	P5	6EC	XMU_0061
Data Mining Techniques	P5	6EC	X_400108
Distributed Algorithms	P5	6EC	X_400211
Energy-efficient Edge Computing	P5	6EC	XMU_0056
ICT for the Global South	P5	6EC	XM_0142
Secure Computation	P5	6EC	XMU_0065
Software Testing	P5	6EC	X_400439
AI for Security	P6	6EC	XM_0178
Inclusive ICT	P6	6EC	XM_0177
Machine Learning for the Quantified Self	P6	6EC	XM_40012

Computer Science specialisation Software Engineering & Green IT constrained choice

Description

The curriculum of the Master Computer Science consists of several constrained choice categories, together ensuring the breadth of the study plan.

Programme Sections

- CS constrained choice FCC core
- CS constrained choice BDE core
- CS constrained choice CSI core
- Computer Science JD Mathematics constrained choice year 1 and year 2
- Computer Science JD Societal Perspectives on Computer Science constrained choice
- Computer Sciences JD Security constrained choice
- CS constrained choice SEG core

CS constrained choice FCC core

Courses

Course Name	Period	Credits	Code
Logical Verification	P1	6EC	XM_0167
Advanced Algorithms	P2	6EC	XMU_0060
Foundations of Cryptography	P2	6EC	XM_0174
Information Theoretic Learning	P2	6EC	XMU_0064
Advanced Logic	P4	6EC	X_405048
Term Rewriting Systems	P4	6EC	XM_400121
Advanced Graph Algorithms	P5	6EC	XM_0173
Distributed Algorithms	P5	6EC	X_400211

CS constrained choice BDE core

Courses

Course Name	Period	Credits	Code
Digital Twin Engineering	P1	6EC	XMU_0068
Data Preparation	P3	6EC	XMU_0058
Data Protection Technologies	P4	6EC	XMU_0057
Web Services and Cloud-Based Systems	P4	6EC	XMU_418110
Data Mining Techniques	P5	6EC	X_400108

CS constrained choice CSI core

Courses

Course Name	Period	Credits	Code
Advanced Operating Systems	P1	6EC	XM_40014
Programming Large-Scale Parallel Systems	P1	6EC	XM_40017
Distributed Systems	P2	6EC	X_400130
Parallel Programming Practical	P3	6EC	X_400162
Multi-core Processor Systems	P4	6EC	XMU_0055
Accelerator-Centric Computing Ecosystems	P5	6EC	XM_0171
Energy-efficient Edge Computing	P5	6EC	XMU_0056

Computer Science JD Mathematics constrained choice year 1 and year 2

Courses

Course Name	Period	Credits	Code
Dynamic Programming and Reinforcement Learning	P2	6EC	XM_0093
Performance of Networked Systems	P2	6EC	X_405105
Experimental Design and Data Analysis	P4	6EC	X_405078

Computer Science JD Societal Perspectives on Computer Science constrained choice

Courses

Course Name	Period	Credits	Code
E-Commerce Law	P3	6EC	R_E.commerc
History of Digital Cultures	P3	6EC	XM_0134
Entrepreneurship in Analytics and AI	P4	6EC	XM_0090
ICT for the Global South	P5	6EC	XM_0142
Inclusive ICT	P6	6EC	XM_0177

Computer Sciences JD Security constrained choice

Courses

Course Name	Period	Credits	Code
Software Security	P1	6EC	X_400127
Verification for Security	P1	6EC	XM_0099
Network Security	P2	6EC	XM_0100
Secure and Trustworthy Machine Learning	P3	6EC	XM_0179
Security Experiments and Measurements	P3	6EC	XM_0098
Cryptographic Engineering	P4	6EC	XMU_0047
Secure Computation	P5	6EC	XMU_0065
AI for Security	P6	6EC	XM_0178

CS constrained choice SEG core

Courses

Course Name	Period	Credits	Code
Digitalization and Sustainability	P1	6EC	XM_0089
Green Lab	P1	6EC	X_418158
Service Oriented Design	P1	6EC	X_405061
Digital Architecture	P2	6EC	XM_0127
Fundamentals of Adaptive Software	P2	6EC	XM_0128
Software Language Engineering	P4	6EC	XM_0172
Software Testing	P5	6EC	X_400439

Courses

Accelerator-Centric Computing Ecosystems

Course Code	XM_0171
Credits	6
Period	P5
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. T. De Matteis
Examiner	dr. T. De Matteis
Teaching Staff	dr. D. Bonetta, dr. T. De Matteis
Teaching method(s)	Seminar, Lecture

Course Objective

With the end of Moore's law, the need for continuing performance scaling has led to the emergence of heterogeneous computing systems infrastructures. Alongside traditional multicore CPUs, we now see GPUs (Graphic Processing Units), FPGAs (Field Programmable Gate Arrays), and other domain-specific architectures (such as DPUs and TPUs) being commonly offered as accelerators to speed up part of applications. Each of these devices has different performance, power requirements, and parallelism opportunities that must be understood to leverage the diversity of modern and future ICT infrastructure.

This course aims to equip students with an in-depth understanding of accelerated computing systems. This involves discussing diverse computing architectures, including alternatives to traditional ("von Neumann") computer organization, and analyzing their different programming models and optimization techniques, developing a comprehensive understanding of the intricacies and opportunities of each presented approach.

After completing this course, you will:

- Understand diverse computing architectures (e.g., GPUs, FPGAs, and emerging paradigms) and relate them

to classical computer organization [Knowledge and understanding]
- Acquire a comprehensive knowledge of the parallel programming models that characterize different compute accelerators and optimization techniques for diverse workloads. [Knowledge and understanding]
- Develop the ability to design, analyze, and implement parallel algorithms for specific computing architectures [Apply Knowledge]
- Evaluate and make informed decisions about the selection and optimization of algorithms for diverse computing architectures, considering factors such as performance, energy efficiency, and scalability [Apply Knowledge] [Making judgements]
- Effectively communicate technical concepts related to advanced computing systems [Communication]
- Develop learning skills to stay informed about and critically analyze emerging technologies in advanced computing systems [Lifelong learning skills]

Course Content

The course will consist of weekly lectures, lab sessions, and practical programming assignments.

We will start by discussing the challenges faced by the computing community, such as the end of Dennard scaling and Moore's law, as well as the emergence of new computational-hungry domains – e.g., AI. We will discuss how these challenges have led to the development and diffusion of heterogeneous computing systems. We will introduce general-purpose graphics accelerators (GPGPUs) using, for example, NVidia's programming model CUDA. We will present dataflow architectures (particularly FPGAs) as an alternative to classical von Neumann architectures. We will discuss their programming challenges and how to develop efficient FPGA programs using High-Level Synthesis tools. Finally, we will conclude by discussing novel/research computing architectures and the current research directions in this field.

Additional Information Teaching Methods

2 Lectures per week, Practical coding group assignments with 1 lab session per week for training/QA (deliverables: code and report).

Method of Assessment

There are two components in this course: Exam (50% of the final grade) and Practical Assignments (50% of the final grade). Resit is possible only for the exam. Exam and practical assignments must be passed separately to pass this course.

Practical assignments are done in small teams. Students are graded on the completion of the assignment and their understanding of the proposed solution (based on the oral discussion/interview and report/code submitted).

[FRAUD] The course is made fraud-resistant by having specialized group projects with interviews to assess the knowledge of the students. Furthermore, the exam is taken individually in person on campus.

Please note that your grade will be registered as follows:

- If you did not submit any assignments and did not participate in the exam, the result will be registered as No Show (NS).
- If you did not meet the minimum grade requirements for the assignments or the exam, but your weighted average is >= 5.5, your grade is NVD.
- Otherwise, your final grade is your weighted average of the two components, rounded to half points except for 5.5.
- There is no resit opportunity for the practical assignments. Results for practical assignments and for the exam can not be transferred from previous years.

Literature

The course does not follow a specific textbook.

Slides and further online reading material/links will be provided via Canvas.

Additional Information Target Audience

MSc Computer Science

Recommended background knowledge

Computer organization (CPU caches, instructions, multicores), knowledge of C/C++, and parallel programming. For practical assignments: Linux development environment - gcc, make, gdb, ssh.

Advanced Algorithms

Course Code	XMU_0060
Credits	6
Period	P2
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	J.V.A. Tielen MA
Examiner	
Teaching Staff	
Teaching method(s)	

Course Objective

This is an UvA course. Please follow the link for the course description.

Course Content

https://studiegids.uva.nl/xmlpages/page/2025-2026-en/search-course/course/130403

Additional Information Teaching Methods

-

Method of Assessment

-

Custom Course Registration

This course is offered by the UvA. For more information contactFNWI Education Service Centre.

Enrolment via https://glass.uva.nl is required.

Advanced Graph Algorithms

Course Code	XM_0173
Credits	6
Period	P5
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. Y. Nazari MSc
Examiner	dr. Y. Nazari MSc
Teaching Staff	dr. Y. Nazari MSc
Teaching method(s)	Seminar, Lecture

Course Objective

The goal of this course is to use techniques and tools in algorithms and theoretical computer science and apply them to the domain of graph algorithms. Graph algorithms provide the fundamental primitives for a wide range of applications in computer science from network routing, to data analytics, computing systems, machine learning, etc. A deeper and systematic understanding of these algorithms will help students develop their problem solving, algorithmic and analytical skills and to apply these skills to solving computational tasks in various areas. In addition to the fundamentals of graph algorithms, this course will also cover some of the recent algorithmic trends. In particular, we discuss graph algorithms in classic and modern computational models that represent distributed and parallel settings. This course also provides the foundations for research in algorithms and combinatorial optimization.

- Efficient algorithms for fundamental computational tasks on graphs (knowledge and understanding)

- Modelling computational tasks as graph problems (understanding and applying knowledge)
- Formal analysis of various computational resources and models (understanding)
- Using optimization techniques for solving hard problems (knowledge and understanding)
- Theoretical modeling graph analysis tasks on modern computational systems (understanding)

Course Content

Problems related to reachability, shortest-paths, maximum flow and bipartite matching in classic and distributed/parallel settings. Algebraic approaches (matrix multiplication) for graph algorithms and probabilistic algorithms. Computationally hard graph problems and applying optimization techniques such as local search, integer programming, etc.

Additional Information Teaching Methods

There will be two in-person lectures per week. In addition there will be weekly interactive problem solving sessions.

Method of Assessment

The final grade will be based on one final exam (8 points out of 10) and several assignments (2 points out of 10). Each assignment gets a fail or pass grade, and students get a grade out of 2 based on the number of passed assignments. In other words, the final grade is 0.8* (score in the final exam)+ 0.2*(number of passed assignments). There will be one re-sit replacing the exam portion of the grade (8 points), but there will be no resit for the assignment portion of the grade.

Literature

Reading material (including online resources such as lecture notes and textbooks) will be provided via canvas.

Additional Information Target Audience

Masters in Computer Science

Recommended background knowledge

Data Structure and Algorithms, Discrete Mathematics, Basics of Probability Theory and experience with mathematical proofs is required. Further recommended background: Linear Algebra.

Advanced Logic

Course Code	X_405048
Credits	6
Period	P4
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. F. van Raamsdonk
Examiner	dr. F. van Raamsdonk
Teaching Staff	dr. F. van Raamsdonk
Teaching method(s)	Seminar, Written partial exam, Lecture

Course Objective

The objective of the course Advanced Logic is to obtain a basic understanding of modal logic.

Knowledge and insight:

- know the Kripke semantics, and variations on that definition, for example game semantics,
- know the tableaux and sequent method for decidability,
- know the notion of bisimulation, and the game approach to (dis)proving bisimularity,
- know the nature of Hilbert style proof systems,
- know basic temporal logic and multi-model logic,
- know propositional dynamic logic and its semantics;

Applying knowledge and insight:

- be able to give proofs using Kripke semantics, game semantics,
- be able to decide on (non)validity using the tableaux or sequents method,
- be able to (dis)prove bisimilarity of two pointed models also in a game approach,
- be able to make elementary derivations in Hilbert style systems,
- be able to give proofs and show (non-)bisimilarity for basic temporal logic and multi-modal logic,
- be able to (dis)prove validity and discuss models in propositional dynamic logic;

Judgement:

- be able to (dis)prove definability of operators and frame properties in basic modal logic and also in basic temporal logic and multi-modal logic,
- be able to discuss some decidability issues;

Communication:

- be able to write elementary proofs;

Learning skills:

- have acquired some flexibility in dealing with different versions of modal logic.

Course Content

The course Advanced Logic is concerned with modal logic and its applications to computer science. We study some themes from the book Modal Logics for Open Minds by Johan van Benthem, in particular: basic modal logic and possible world semantics, bisimulation and invariance, modal definability and expressive power, decidability, proof systems, propositional dynamic logic, some epistemic logic.

Additional Information Teaching Methods

We have two lectures per week, for the duration of seven weeks. Twelve lectures discuss material and two lectures are meant for questions and wrap-up.
We have one exercise class per week, for the duration of seven weeks. We discuss (on paper) exercises.

Method of Assessment

There is a written closed-book midterm exam about the first half of the material in week 4 of the course.
There is a written closed-book final exam about all material in week 8 of the course.

If the grade for the midterm exam is higher than the grade for the final exam, then the midterm counts for 20% of the final grade (midterm-grade + 4*final-exam-grade)/5), otherwise the final grade is the grade of the final exam.

Literature

- Johan van Benthem, Modal Logics for Open Minds, CSLI Publications 2010.
- In addition we use slides and exercise sheets.

Additional Information Target Audience

MSc Artificial Intelligence
MSc Computer Science
MSc Computer Security

Explanation Canvas

We use Canvas for the schedule of the course, and to make the slides for the lectures and the exercises for the exercise classes available.
We use Canvas further to provide feedback to the homework assignments, to give the grade for the midterm, and for announcements.

Recommended background knowledge

Basic knowledge of the syntax and semantics of first-order proposition and predicate logic, as for example taught in the Bachelor course Logic and Modelling (X_401015).

Advanced Operating Systems

Course Code	XM_40014
Credits	6
Period	P1
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. C. Giuffrida
Examiner	prof. dr. C. Giuffrida
Teaching Staff	dr. E. van der Kouwe, prof. dr. C. Giuffrida
Teaching method(s)	Lecture

Course Objective

Gaining a deep understanding of design and implementation of major OS subsystems.

Upon completion of this course, students will:

- have knowledge of advanced operating system design and implementation principles adopted by mainstream operating systems such as Linux (Knowledge & Understanding)
- have a basic understanding of recent trends in operating systems research (Lifelong learning skills)
- be familiar with design and implementation principles of physical memory management (Knowledge & Understanding, Applying Knowledge & Understanding, Making Judgements, Communication)
- be familiar with design and implementation principles of page table management (Knowledge & Understanding, Applying Knowledge & Understanding, Making Judgements, Communication)
- be familiar with design and implementation principles of process management (Knowledge & Understanding, Applying Knowledge & Understanding, Making Judgements, Communication)
- be familiar with design and implementation principles of virtual memory management (Knowledge & Understanding, Applying Knowledge & Understanding, Making Judgements, Communication)
- be familiar with design and implementation principles of multiprocessing (Knowledge & Understanding, Applying Knowledge & Understanding, Making Judgements, Communication)
- be familiar with design and implementation principles of multicore management (Knowledge & Understanding, Applying Knowledge & Understanding, Making Judgements, Communication)
- be familiar with design and implementation principles of page reclaiming (Knowledge & Understanding, Applying Knowledge & Understanding, Making Judgements, Communication)

Course Content

The course will feature a number of hands-on assignments accompanied by lectures on advanced operating system kernel design and programming concepts. In each assignment, students will be expected to start with a minimal kernel implementation and exercise their kernel hacking skills on one of the major operating subsystems (i.e., memory management, process management, drivers, etc.). This will involve programming in both C and assembly as well as directly interfacing with the hardware. The course will also link lectures and assignments to modern operating system features and offer insights into state-of-the-art OS research efforts.

Additional Information Teaching Methods

Lectures and practical assignments.

Method of Assessment

Weekly examination based on practical assignments and lectures. Each week, the student can get up to 11 points for the weekly examination: 6 points from a practical assignment, 2 points for the bonus parts of the assignment, and 3 points from an oral discussion that is based on the assignment and its companion lectures for that week. In each of the first six weeks, you need at least 50% of the points for the oral discussion (at least 1.5 points), along with passing all basic compliance tests for the practical assignment. The seventh (and last) practical assignment is special: the oral discussion is replaced by a written report and there is no minimum grade to pass the assignment. The final grade is the average of all weekly examinations' grades (including the last practical assignment grade, which is considered to be zero if it was not submitted).

Please note that your final grade will be registered as follows:

- If you do not submit any assignment and do not participate in any oral discussion, you will receive a NS.
- In all other cases, you will receive either a numeric grade or incomplete (NVD).
- If you did not meet the minimum grade requirements for the assignments, your grade is NVD.
- Otherwise, your final grade is your weighted average.
- If you plan to drop the course, do not submit anything, as NVD is administratively considered a failing grade.

There is no resit opportunity for the practical assignments, nor for the oral discussions. Results for practical assignments and for the oral discussion may be transferred from previous years if their contents did not change significantly since the result was obtained (as determined by the examiner). If you submit an assignment or participate in the oral discussion, the new result replaces any transferred results.

We ensure that each student individually demonstrates their abilities through oral exams to test understanding of the assignments, through active plagiarism detection for the assignments, and by investigating whether each group member contributed sufficiently for the group assignment.

Literature

Slides and online material.

Additional Information Target Audience

MSc Computer Security
MSc Computer Science

Recommended background knowledge

Students are expected to have completed a Bachelor-level Operating systems course as well as to have experience in C/C++ programming and knowledge of assembly and computer organization.

AI for Security

Course Code	XM_0178
Credits	6
Period	P6
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. M. Zhang
Examiner	dr. M. Zhang
Teaching Staff	M. Chen, dr. M. Zhang
Teaching method(s)	Seminar, Lecture

Course Objective

This course explores a range of topics at the intersection of security and artificial intelligence (AI). The primary goal is to develop students' ability to think critically about how AI techniques can be effectively applied to address real-world security problems.

Upon completion of this course, the students will:

- Understand several security problems, e.g., software vulnerability mining, vulnerability assessment, anomaly detection (K&U, LS, C)
- Understand AI techniques, e.g., neural networks, large language models (K&U, LS)
- Have knowledge of example(s) about how to apply AI to a domain-specific problem, e.g., model security events into sequential data, connect security incidents as graphs (AK&U, MJ, LS)
- Have developed critical thinking around AI and security (MJ, LS, C)

The acronyms above refer to the Dublin Descriptors covered by each learning objective, specifically: Knowledge and Understanding (K&U), Applied Knowledge and Understanding (AK&U), Making Judgment (MJ), Learning Skills (LS), and Communication (C).

Course Content

This course explores a range of rapidly evolving security topics that have seen significant advances through the application of artificial intelligence (AI) techniques. Topics may include, but are not limited to, AI-assisted software development, automated bug discovery, automated software repair, and vulnerability assessment.

The course is structured around concrete security problems. For each topic, we begin by introducing the core security challenge, then examine how it has been traditionally addressed. We subsequently explore how AI techniques have been applied to enhance or automate solutions to the problem. Finally, we discuss the open challenges and limitations that must be addressed before AI can be effectively applied to new or emerging security

scenarios.

To understand state-of-the-art methods, the course focuses on the discussion of scientific papers published at top-tier venues. Some topics may be accompanied by practical implementations; therefore, programming skills are expected.

Additional Information Teaching Methods

The course is organized into weekly lectures accompanied by practical quizzes and one final report based on a selection of five research papers. A list of papers will be provided, from which each student will choose five. The report should present a critical analysis of a specific security issue addressed in the selected papers, comparing the strengths and limitations of the approaches used. Students are expected to demonstrate critical thinking by identifying open challenges and proposing possible future directions related to the problem.

Peer Review of Reports:

Students will participate in a peer-review process by evaluating reports submitted by their classmates. Each student will provide a detailed assessment based on predefined criteria, including the quality of analysis, clarity of writing, comparison of methods, and depth of critical thinking.

Self-Evaluation of Report:

In addition to peer review, students will complete a self-evaluation of their own report. This will involve providing a detailed self-assessment against the same grading criteria, encouraging reflection on their work and the learning process.

Method of Assessment

The overall grade is determined as follows:

- Quizzes (30%)

Three quizzes, each worth 10%.

- Report (20%)

Report content: 15%, assessed based on depth of analysis, clarity, comparison of methods, and critical insight.
Peer review quality: 5%, based on the thoroughness and constructiveness of the peer reviews provided.

- Final Exam (50%)

A comprehensive exam covering the full course content.

Pass or fail
Quality of the reviews. Additional points will be deducted from students who write poor (e.g., too generic, such as "it is good") or unfair reviews.

To pass the course, you must satisfy the following criteria:

- You must score at least 6.0 on the final exam, and
- You must score 6.0 in total (i.e., with all three components taken together).

Resit
There is a resit for the final exam (50%). You will have to score 6.0 to pass the resit.
It is ***not possible*** to resit the assignments and the peer reviews.

Please note that your grade will be registered as follows:

For the first exam date: if you did not submit any quizzes, report, and did not participate in the exam, the result will be registered as no show (NS). For the second exam date: if you did not participate in the resit exam, the result will be registered as no show (NS). In all other cases, you will receive either a numeric grade or incomplete (NVD).

If you did not meet the minimum grade requirements for the exam (6.0), but your weighted average is >= 6.0, your grade is NVD. Otherwise, your final grade is your weighted average, rounded to half points except for 6.0.

If you plan to drop the course, do not submit anything, as NVD is administratively considered a failing grade.

Entry Requirements

While there are no formal entry requirements, however, students are expected to have foundational knowledge in either cybersecurity or artificial intelligence to fully benefit from the material.

Literature

No set book. All materials (i.e., papers, slides) will be made available during the course.

Additional Information Target Audience

Master Computer Security
Master Computer Science
Master Artificial Intelligence

Additional Information

Please see the information on Canvas.

Explanation Canvas

The submission of the report and the peer review of the report will be done in Canvas/FeedbackFruit.

Because of the way the system works by automatically assigning reviewers, it is not possible to grant any deadline extension for the submission of the reports or the submission of the reviews.

Students should therefore carefully plan their activities and submit also a draft version of their reports well in advance of the deadline (multiple submissions are possible) rather than waiting for the last moment.

Recommended background knowledge

While there are no formal entry requirements, however, students are expected to have foundational knowledge in either cybersecurity or artificial intelligence to fully benefit from the material. Programming skills are also expected.

Algorithmic Game Theory

Course Code	XMU_0063
Credits	6
Period	P1
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	J.V.A. Tielen MA
Examiner	
Teaching Staff	
Teaching method(s)	

Course Objective

This is an UvA course. Please follow the link for the course description.

Course Content

https://studiegids.uva.nl/xmlpages/page/2025-2026/zoek-vak/vak/130040

Additional Information Teaching Methods

-

Method of Assessment

-

Custom Course Registration

This course is offered by the UvA. For more information contactFNWI Education Service Centre.

Enrolment via https://glass.uva.nl is required.

Computational Complexity

Course Code	XMU_0061
Credits	6
Period	P5
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	J.V.A. Tielen MA
Examiner	
Teaching Staff	
Teaching method(s)	

Course Objective

This is an UvA course. Please follow the link for the course description.

Course Content

https://studiegids.uva.nl/xmlpages/page/2025-2026-en/search-course/course/130025

Additional Information Teaching Methods

-

Method of Assessment

-

Custom Course Registration

This course is offered by the UvA. For more information contactFNWI Education Service Centre.

Enrolment via https://glass.uva.nl is required.

Cryptographic Engineering

Course Code	XMU_0047
Credits	6
Period	P4
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	J.V.A. Tielen MA
Examiner	
Teaching Staff	
Teaching method(s)	

Course Objective

This is an UvA course. Please follow the link for the course description.

Course Content

https://studiegids.uva.nl/xmlpages/page/2024-2025-en/search-course/course/119769

Additional Information Teaching Methods

-

Method of Assessment

-

Custom Course Registration

This course is offered by the UvA. For more information contactFNWI Education Service Centre.

Enrolment via https://glass.uva.nl is required.

Data Mining Techniques

Course Code	X_400108
Credits	6
Period	P5
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. M. Hoogendoorn
Examiner	prof. dr. M. Hoogendoorn
Teaching Staff	dr. S. Yu, prof. dr. M. Hoogendoorn, B. Borsboom
Teaching method(s)	Computer lab, Lecture

Course Objective

The aim of the course is that students acquire data mining knowledge and skills that they can apply in a business environment. More precisely, the following learning goals are distinguished:

1. Understand the data mining process
2. Understand exploratory data analysis
3. Understand feature engineering
4. Understand classification techniques
5. Understand regression techniques
6. Understand association rules and recommender systems
7. Understand big data engineering and big data infrastructures
8. Understand current state of the art in data mining
9. Understand ethical concerns and evaluation of results
10. Learning to argue for choices (rationale)

The course if focused on the Dublin descriptors Applying Knowledge and Understanding (since the aim is to improve the practical skills); Making judgements (which technique is appropriate, how to best apply Data Mining for a specific case); Communication skills (how to report on your approach, choices and your results), and Learning skills (able to find new relevant techniques, assess their suitability, etc.).

How the aims are to be achieved: Students will acquire knowledge and skills mainly through the following: an overview of the most common data mining algorithms and techniques (in lectures), a survey of typical and interesting data mining applications, and practical assignments to gain "hands on" experience. The application of skills in a business environment will be simulated through various assignments of the course.

Course Content

The course is intended to introduce Data Mining Techniques to students that are new to the field as well as to more experienced students. The main aim is to gain a more practical perspective towards Data Mining Techniques/Machine Learning. Lectures will cover more basic things for those new to the field (general introduction into Data Mining, classical algorithms such as decision trees, association rules, neural networks, ensemble learning, etc.) and on top will discuss advanced topics including deep learning, recommender systems, big data infrastructures, and text mining. A number of successful applications in the area will also be discussed. In addition to lectures, there will be an extensive practical part, where students will experiment with various data mining algorithms and data sets. The grade for the course will be based on these practical assignments.

Additional Information Teaching Methods

Lectures (h) and practical sessions (pra). Lectures are planned to be interactive: there will be small questions, etc.

Method of Assessment

Practical assignments. There will be two assignments done in groups of three. For the first assignment there is a choice: going for a basic version of the assignment (suited for those new to the domain) or a more advanced one (for students with more experience). The second assignment is the same for all and will involve an in-class Kaggle competition. The first assignment counts for 40% and the second for 60%.

An interview will take place on an individual student level to make sure that (1) each student understands the content of the graded material, and (2) has made a sufficient contribution to the group work. Only when this check is passed will a final grade be given. Furthermore, the grade of both assignments needs to be sufficient to pass the course. Finally, attendance of the final lecture (in which the final ranking of the Kaggle competition is announced and selected groups of students will be asked to present their solution) is mandatory as it contributes to the learning objectives of the course.

There is an option to resit one of the assignments when the assignment has been submitted but an insufficient grade is obtained (only when the other assignment grade is sufficient). This involves improving your assignment to at least a sufficient level within two weeks after having received your grade for the second assignment. The maximum grade of the resit is a 6.0.

Literature

Optional: Ian H. Witten, Eibe Frank, Mark A. Hall, and Christopher J. Pal. Data Mining: Practical Machine Learning Tools and Techniques (Fourth Edition). Morgan Kaufmann, 2016 978-0128042915.

Additional Information Target Audience

Master Artificial Intelligence, Master Computer Science, Master Bioinformatics and Systems Biology, Master Business Analytics

Recommended background knowledge

1. Statistics
2. Programming Skills

Data Preparation

Course Code	XMU_0058
Credits	6
Period	P3
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	J.V.A. Tielen MA
Examiner	
Teaching Staff	
Teaching method(s)	

Course Objective

This is an UvA course. Please follow the link for the course description.

Course Content

https://studiegids.uva.nl/xmlpages/page/2025-2026-en/search-course/course/130406

Additional Information Teaching Methods

-

Method of Assessment

-

Custom Course Registration

This course is offered by the UvA. For more information contactFNWI Education Service Centre.

Enrolment via https://glass.uva.nl is required.

Data Protection Technologies

Course Code	XMU_0057
Credits	6
Period	P4
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	J.V.A. Tielen MA
Examiner	
Teaching Staff	
Teaching method(s)	

Course Objective

This is an UvA course. Please follow the link for the course description.

Course Content

https://studiegids.uva.nl/xmlpages/page/2025-2026-en/search-course/course/130401

Additional Information Teaching Methods

-

Method of Assessment

-

Additional Information

This course is offered by the UvA. For more information contactFNWI Education Service Centre.

Enrolment via https://glass.uva.nl is required.

Digital Architecture

Course Code	XM_0127
Credits	6
Period	P2
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. R.C. de Boer
Examiner	dr. R.C. de Boer
Teaching Staff	dr. R.C. de Boer, dr. R.C. de Boer
Teaching method(s)	Seminar, Lecture

Course Objective

Knowledge and understanding
At the end of the course, students will have gained knowledge about architecturally significant requirements, architecture representation, design, quality and analysis. They understand the drivers behind architectural decisions and the ‘wicked’ nature of architecture design, and appreciate the role of the architect.

Applying knowledge and understanding
Students will be able to reason about the architecture of non-trivial software-intensive systems, and apply their

knowledge and understanding to a concrete (fictitious) case that involves many stakeholders with different needs and concerns.

Making judgments
Students will make their own architectural decisions, and become aware of the rationale involved in making trade-offs and choosing design options among alternatives. They learn to appreciate the design decisions proposed by others. In a serious game setting, students will further experience architectural design decision making.

Communication skills
Students present their progress to the other course participants, and engage in debate about their architecture design. They communicate their final architecture design, including the architectural decisions and their rationale, in a document.

Learning skills
Students have to coordinate and negotiate team work, and manage a significant workload in the allotted time. Through searching and integrating background material, students build a progressive understanding of the 'problem domain' relevant to the case.

Course Content

Students work in groups to design an architecture for a fictitious software-intensive system. They have to develop different representations (called views) of the architecture. These different representations emphasize different concerns of people or organizations that have a stake in the system. The group work is complemented by theory lectures and in-depth sessions.

Additional Information Teaching Methods

Teaching methods and contact hours:

- Lectures: two hours per week
- Group work with a number of assignments; two contact hours per week
- Tutorials and in-depth sessions: two hours per week.

Attendance of the first lecture is compulsory! Students are required to participate in group work. All teams must be present at their scheduled contact hours to present their progress and engage in debate with the other teams.

Note that this course is fully taught on campus. Remote participation is not possible.

Method of Assessment

Graded assignments

Project assignments (group assignment): There are two assignments based on a practical case. These assignments are done as group work. The average grade of the assignments should be ≥ 5.5 and will account for 75% of the final grade. Your work will be evaluated by means of a rubric published on Canvas.

Written exam (individual assignment): To complete the course you will have to take part in the written exam for Digital Architecture. The exam takes place in the exam week at the end of the period. Your grade for the exam should be ≥ 5.5 and will account for 25% of your final grade.

Pass/fail-assigments

Participation in the serious game and Game survey (individual assignment): In-person participation in the game is compulsory. The game will be played by all students simultaneously on campus. After playing the architecture design decision-making game, each student should fill out the game survey individually. There will be only one opportunity to play the game, it is not possible to retake this assignment on another occasion. The assignment will be graded as "pass" or "fail".

Practical sessions: attendance (group assignment) and presentation (individual assignment): Participation in the practical sessions is compulsory. Every team must be present each week and present their progress and engage with other teams, and all team members are expected to be there. Every team member must present the team's progress at least once. The assignment will be graded as "pass" or "fail".

Weekly progress report (individual assignment): We ask you to – individually – complete a simple weekly survey to indicate your team's progress, collaboration and communication and the participation of all team members (including yourself). The assignment will be graded as "pass" or "fail".

Compensation and final grade

Both the exam grade and the weighted average of the project assignment grades MUST be 5.5 or higher. Compensation between the two is not possible. Additionally, all "pass"/"fail"-assignments and presentations must be "pass" in order to pass the course.

- In case any of the pass/fail-assignments is a 'fail', the final grade of the course will be 'NVD', otherwise:
- In case any of the graded assignments is insufficient, the final grade of the course will be that insufficient grade, otherwise:
- In case all pass/fail-assignments are 'pass' and all graded assignments are at least 5.5, the final grade of the course will be the weighted average of the project assignment grade (75%) and the exam grade (25%).

Mode of re-examination

A failed project assignment can be revised and resubmitted once; a failed presentation can be retried once; failing or missing the game assignment CANNOT be retried. A failed exam can be retried in the resit period.

Literature

Len Bass et al., Software Architecture in Practice, 4th edition, 2021

Additional Information Target Audience

Master Information Sciences
Master Computer Science

Custom Course Registration

Group enrolment takes place in Canvas.

Explanation Canvas

Further information for this course will be made available online (Canvas). All students must be enrolled in the course Canvas community.

Recommended background knowledge

Software design.

Digital Business Ecosystems and Platforms

Course Code	XM_0176
Credits	6
Period	P4
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. J. Gordijn
Examiner	dr. J. Gordijn
Teaching Staff	dr. J. Gordijn
Teaching method(s)	Seminar, Lecture

Course Objective

An ecosystem is a collection of parties who work cooperatively and competitively to satisfy customer needs. Well-known examples are Google, Facebook, and Netflix, but there are many other eco-systems in the field, e.g. in Fintech, energy, and automotive industry. In this course, we focus on digital ecosystems which are enabled by Internet- and related technologies. Frequently, these ecosystems are disrupted by innovative information technologies, such as machine learning, blockchain, and 5G mobile. The overall goal of this course is to analyze disruptive opportunities for an eco-system given a particular emerging technology. This year, that technology is blockchain.

Upon completion of this course, students will:

- have gained knowledge and understanding of the concept of ‘eco-system’ (Knowledge and understanding);
- have gained knowledge and understanding of various methods for designing and analyzing eco-systems and

their e-services, including e3value, trust modelling, BPMN, and UML class modelling (Knowledge and understanding);
- have gained knowledge and understanding of the selected technology of this year in order to analyze, design and implement a disruptive eco-system (Knowledge and understanding);
- be able to analyze and design an eco-system offering an e-service from different viewpoints (Applying knowledge and understanding);
- be able to elicit different e-service (business) ideas that can be provided in an eco-system (Applying knowledge and understanding);
- be able to explore the e-service idea from a business perspective. In this way, the student will be able to further design and develop the e-service idea using different analysis techniques (Applying knowledge and understanding);
- be able to transform the business perspective of the e-service design into a design reflecting the IT perspective (Applying knowledge and understanding);
- be able to implement (part of) the e-service using the selected disruptive technology (Applying knowledge and understanding);
- be able to make informed decisions about eco-systems and their e-services based on possible risks and opportunities (Making judgments);
- be able to assess the quality and correctness of the various conceptual models created (Making judgments);
- be able to pitch a new idea for an eco-system/e-service to investors (Communication skills);
- be able to concisely present the selected solution (Communication skills);
- be able to write a scientifically well-founded report about the eco-system/e-service under development in a group of students (Communication skills);
- have learned to work in a project group and keep each other accountable for agreed results (Learning skills);
- have learned to improve themselves based on various feedback mechanisms (Learning skills).

Course Content

The course comprises the following topics:

- Eco-systems and platforms
- Introduction in the selected technologyof this year
- Business models and value proposition design
- The e3value method for understanding business networks
- The Business Process Modelling Notation for analysing cross-organizational networks
- The Unified Modelling Language – class modelling
- Transaction modelling
- Technological design issues

Additional Information Teaching Methods

The following methods are used:

- Lectures twice a week. These lectures include in-class workshops to practice the various methods used in this course
- Group work: teams of about 2/3 students design an eco-system. At the beginning of the course a number of eco- systems are offered as inspiration
- Group feedback sessions: Each week, students will get personal feedback in face-to-face feedback sessions. Moreover, groups will receive feedback on their submitted reports by email.

Method of Assessment

Individual exam on the theory (50%), Teamwork project assignments (50%).

To pass the course, the scores of each component must be 5.5. or higher.
Compensation: it is not possible to compensate one component with another.
Mode of re-examination: if insufficient, each component can be revised and resubmitted once.

Literature

A (selection of a) list of papers on the course topics. Material distributed on Canvas.

Additional Information Target Audience

Master Artificial Intelligence
Master Computer Science
Master Information Sciences

Custom Course Registration

Further information for this course will be made available online (Canvas).
All students must be enrolled in the course Canvas community. Group enrolment takes place in Canvas.

Recommended background knowledge

The student should have sufficient knowledge about Requirements Engineering and Conceptual Modelling and should master such a modelling method (at least UML class or activity diagrams).

Digital Twin Engineering

Course Code	XMU_0068
Credits	6
Period	P1
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	J.V.A. Tielen MA
Examiner	
Teaching Staff	
Teaching method(s)	

Course Objective

This is an UvA course. Please follow the link for the course description.

Course Content

https://studiegids.uva.nl/xmlpages/page/2025-2026/zoek-vak/vak/130422

Additional Information Teaching Methods

-

Method of Assessment

This course is offered by the UvA. For more information contact FNWI Education Service Centre.

Enrolment via https://glass.uva.nl is required.

Digitalization and Sustainability

Course Code	XM_0089
Credits	6
Period	P1
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. P. Lago
Examiner	prof. dr. P. Lago
Teaching Staff	prof. dr. P. Lago
Teaching method(s)	Seminar, Lecture

Course Objective

At the end of the course, the students will:

- be familiar with basic knowledge about digitalization and digital transformation, sustainability, and the role of digitalization in achieving business- and other sustainability goals within society and organizations (Knowledge and understanding; Applying knowledge and understanding).

- be able to reason about the technology- and business-related digitalization and sustainability concerns, and apply their reasoning to a concrete project (Applying knowledge and understanding; Making judgments).
- have a basic understanding of the types of sustainability impacts of digital solutions, so to identify and assess the trade-offs between the different sustainability concerns addressed by digital solutions (Knowledge and understanding; Applying knowledge and understanding; Making judgments).
- be able to write a scientific report about a concrete digitalization-and-sustainability project in a group of students (Applying knowledge and understanding; Making judgments; Communication skills).
- be trained to (i) explore the problem- and solution space in the digital transition of a specific sector/domain, and (ii) identify and address a set of relevant sustainability goals (Applying knowledge and understanding; Making judgments; Learning skills).
- be able to reflect critically on what learned, its impact in society, and express own motivated opinions (Applying knowledge and understanding; Making judgments; Communication skills; Learning skills)

Course Content

The course follows a mixed-method approach and includes live lectures, flipped-classes with video lectures, active discussions, and teamwork.

The lectures explain the basic concepts related to digitalization (such as the notions of digitalization and digital transformation, the role of technology, and the impact on business and society) and sustainability (such as the notions of sustainable software and software for sustainability, how to frame sustainability-quality concerns in the design of digital solutions, and how to assess sustainability impacts).

Discussions are based on the video/live lectures and a set of papers the students will reflect upon.

The students participate in small teams to incrementally develop an understanding of the digital transformation of a selected sector/domain and the related sustainability concerns, and work on a shared project and associated report.

Additional Information Teaching Methods

The course will be centered on the practical project (in teamwork) of analyzing the sustainability-, business- and technology- implications of digitalizing a sector or application domain of own choice. The project is generic by design, to allow the students reflect on the gaps and opportunities for digital transformation, and choose their own focus for the most promising digitalization and sustainability goals.

The theory will be explained in lectures (3 on digitalization topics and 3 on sustainability topics) some of which may be video-recorded upfront. Each week, live lectures will engage and challenge the students in lively discussions; while for flipped-classes the students will watch the video lecture upfront and individually carry out simple understanding tasks which will be discussed afterward in a live plenary session. The students will also read a paper each, distributed upfront related to each week’s topics, and answer reflective questions that help apply the gained knowledge to the practical project. There will be weekly active discussions in teamwork sessions, where the students will work in small teams on their practical project. Input to the teamwork sessions will be the reflection on the papers the team has read, and the specific project assignment. There is also a significant amount of self-study, both to study the theory, read the papers, and work on the practical project. The scheduled sessions are 2 hours per week lectures/discussion (l), and 2 hours per week teamwork (s).

Attendance is mandatory.

Method of Assessment

Type and Weight of each component: Teamwork project assignments (80%, two group assignments), Individual assignment (20%). As part of the teamwork project assignments, each student gives an individual presentation. To pass the course, the scores of each component must be 6.0 or higher.

Compensation: it is not possible to compensate one component with another.

Mode of re-examination: if insufficient, each component can be revised and resubmitted once.

Literature

A (selection of a) list of papers on the course topics. Course material distributed on Canvas.

Additional Information Target Audience

MSc Information Sciences, MSc Computer Science

Additional Information

The course has a maximum enrolment limit of 100. The students of the Master Information Sciences, for whom it is compulsory, have priority over the students that take this course as an elective or a constrained choice course.

The students that take this course as an elective or a constrained choice course run the risk of not being able to participate, even if they have registered on time, if the maximum enrolment limit has been reached.

The students of the Master Information Sciences, for whom it is compulsory, run the risk of being initially refused if the enrolment limit has been reached at the time of registration. However this will be adjusted manually after registration has been closed so that all the students of the Master Information Sciences will be able to participate.

Custom Course Registration

Further information for this course will be made available online (Canvas). All students must be enrolled in the course Canvas community.

Group enrolment takes place in Canvas.

Explanation Canvas

Instructions about group enrolment and (if applicable) software tools for attendance will be provided at least one week before the course starts. All students must be enrolled before then.

Recommended background knowledge

Basic knowledge of some programming language. Knowledge of UML.

Distributed Algorithms

Course Code	X_400211
Credits	6
Period	P5
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. W.J. Fokkink
Examiner	prof. dr. W.J. Fokkink
Teaching Staff	prof. dr. W.J. Fokkink, dr. Y. Nazari MSc
Teaching method(s)	Seminar, Lecture

Course Objective

The main objective is to develop an algorithmic frame of mind for solving fundamental problems in distributed computing, by obtaining insight into concurrency concepts. The course offers a bird's-eye view of a wide range of algorithms for basic and important challenges in distributed systems.

A key characteristic of the course is that correctness arguments and complexity calculations of distributed algorithms are provided in an intuitive fashion and by means of examples and exercises.

At the end of the course, the student will:

- be acquainted with fundamental aspects of distributed algorithms, notably logical clocks, wave algorithms, snapshots, rollback recovery, consensus, and self-stabilization; (knowledge and understanding)
- be able to apply algorithms for termination and deadlock detection, routing, leader election, and mutual exclusion; (applying knowledge and understanding)
- have a grasp of impossibility results regarding anonymous networks and networks in which nodes can crash; (knowledge and understanding)
- be aware of certain attacks, data structures (notably Merkle trees), and key exchange and authentication protocols to safeguard the security of distributed systems; (knowledge and understanding)
- have a clear understanding of advanced protocols for distributed transactions, cryptocurrencies, and quantum cryptography. (knowledge and understanding)

Course Content

The following topics are treated in the course: Logical clocks, snapshots, graph traversal, termination detection, garbage collection, deadlock detection, routing, election, minimal spanning trees, anonymous networks, checkpointing, fault tolerance, failure detection, consensus, mutual exclusion, self-stabilization, distributed

transactions, dynamic networks, key exchange, quantum cryptography, authentication, digital signatures, blockchains.

Additional Information Teaching Methods

Lectures: 4 hours per week. Exercise classes: 2 hours per week.

Method of Assessment

Written exam. At the exam, a physical copy or print-out of the textbook can be used. All books will be checked by the lecturer at the exam.

Literature

W.J. Fokkink, Distributed Algorithms: An Intuitive Approach (forthcoming 3rd edition, draft), MIT Press, 2025.

A pdf of the textbook is made available on the Canvas page.

Additional Information Target Audience

MSc Computer Science
MSc Computer Security
MSc Econometrics and Operations Research

Recommended background knowledge

Data Structures & Algorithms.

Distributed Systems

Course Code	X_400130
Credits	6
Period	P2
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	ir. J.J.R. Donkervliet MSc
Examiner	prof. dr. ir. A. Iosup
Teaching Staff	ir. S. Talluri, prof. dr. ir. A. Iosup, dr. T. De Matteis, ir. J.J.R. Donkervliet MSc
Teaching method(s)	Seminar, Lecture

Course Objective

By taking this course, you will learn to:

1. Explain the basic concepts, objectives, and functions of distributed computing (eco)systems, e.g., communication, resource management and scheduling, data consistency, fault-tolerance, and performance. (Knowledge and understanding)
2. Compare the basic characteristics of distributed computing with other computing paradigms (i.e., centralized, parallel). (Applying knowledge and understanding) (Making judgements)
3. Identify proficiently the different flavors of modern distributed ecosystems (i.e., peer-to-peer systems, cluster computing, grid computing, cloud computing, datacenters, distributed HPC, SDN, Big Data systems, IoT systems). (Knowledge and understanding) (Applying knowledge and understanding) (Making judgements)
4. Analyze proficiently the trade-offs inherent in the design of modern distributed systems. (Applying knowledge and understanding) (Making judgements)
5. Design your portfolio distributed system, with many basic and some complex operations of modern distributed systems. (Applying knowledge and understanding) (Making judgements)
6. Implement and deploy your portfolio distributed-system. (Applying knowledge and understanding) (Making judgements)
7. Analyze your portfolio distributed-system. (Communication) (Lifelong learning skills)

Course Content

This course focuses on distributed computing systems and ecosystems. In general, debugging and tuning existing systems and ecosystems, and designing, implementing, and analyzing new distributed computing systems remain

vital and challenging for both industry and academia.

Starting with the mid-1990s, computing is undergoing a revolution, in which collections of independent computers appear to users as a single, albeit distributed, computing system. Motivated by the advent of the Internet, the increase in the computation capacity of consumer computers, the commoditization of server-grade machines, energy constraints, etc., the distributed computing paradigm has permeated all fields using computers. Current distributed computing applications range from social networks to banking, from peer-to-peer file-sharing to high-performance computing used in research, from massively multiplayer online games to business-critical workloads, etc. Important advances have helped to fuse heterogeneous resources into truly global distributed systems and ecosystems, for example in scientific computing, where distributed computation is using Big Data and distributed sensors to produce meaningful progress for humankind. We will focus in this course on a number of these modern examples of distributed computing systems.

Although so many distributed systems and ecosystems already exist, the list of conceptual and technical challenges they pose is long. Depending on requirements, even trivial communication between nodes of the distributed system can be challenging. The failure of a single node, or sometimes even a performance hiccup, can bring an entire system down with it, other nodes or entire other systems may also crash, experiencing correlated and catastrophic failures. Data consistency and coordinating nodes remain important challenges made worse by the large scale of real-world deployments. Poor resource management and naive scheduling can lead to orders-of-magnitude higher operational costs and consumption of energy that we simply cannot spare. It is not uncommon for a modern distributed system to quickly rise and then fall in popularity, as exemplified by the 2016 example of Pokemon Go. We will present in this course real-world situations where modern distributed systems have behaved poorly.

Addressing these challenges requires unique approaches and concepts.

Separating concerns and breaking down problems into smaller cases often lead to limited success, because many properties of distributed systems can only be achieved end-to-end. Can anyone imagine a perfectly reliable production pipeline, if even one of its key stages can suffer failures?

Building capability by adding resources is often offset by the distributed nature of the system. Can anyone ignore the physical limitations of communication around the globe? In this course, we will focus on the unique approaches and principles of distributed systems, from specific architectures and communication protocols to specific concepts in resource management and scheduling, data consistency, fault-tolerance, and performance.

Additional Information Teaching Methods

The course uses prof. Iosup's gamification method. The course is taught as a series of lectures, in combination with self-study and design sessions, and with a large practical assignment. The course also has important design and seminar components.

Method of Assessment

Written exam. Report on the practical assignment. In-class assignments include: design, seminar, Q&A. There are other gamification elements.

For a total maximum of 10,000 points (equivalent to a grade of 10), the course activities will be graded as follows:

1. Exam, questions (max 6,000 points)
2. Practical, lab exercise (max 4,000 points)
3. Design and seminar sessions (max 2,000 points)
4. In-class activity: max 50p per session

Other bonuses may be applicable

There is only a resit opportunity for the exam.

Literature

The course uses the following textbook: Maarten van Steen and Andrew S. Tanenbaum, Distributed Systems, 3rd. Ed., online edition, 2017. (free for all) [Online] Available: https://www.distributed-systems.net/index.php/books/distributed-systems-

Recent material is only available as lecture slides and selected publications. The lecture slides also recommend additional literature.

The study guide available on Canvas also indicates other worthwhile sources of information.

Additional Information Target Audience

MSc Computer Science
MSc Computer Security

Additional Information

This course uses the gamification technique developed by prof. Iosup and his team.

Explanation Canvas

The course guide is mandatory reading for every student prior to any other activity in the course.

Recommended background knowledge

Ability to work in teams for the practical assignment. Ability to design software and to develop code using modern software engineering practices, e.g., setting up your own GitHub repository, co-editing using tools such as Overleaf or Sharelatex, etc., is a big plus. Experience with big data processing, multimedia and online gaming systems, scientific (workflow) computing is also a plus.

Students should have taken standard courses on:

- Computer networks
- Operating systems
- Programming paradigms, in particular OOP and/or actor-based approaches
- Software engineering

Prior experience with Internet, web, distributed, or parallel programming and algorithms is helpful.

Prior experience with operating systems development and analysis, and in general experience with computer systems courses, is a big plus.

Dynamic Programming and Reinforcement Learning

Course Code	XM_0093
Credits	6
Period	P2
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. G.M. Koole
Examiner	prof. dr. G.M. Koole
Teaching Staff	V.G.M. Francois Lavet, prof. dr. G.M. Koole
Teaching method(s)	Seminar, Lecture

Course Objective

Dynamic Programming (DP) and Reinforcement Learning (RL) are fields concerned with decision making over time. After completing this course, the student

- is familiar with the commonly used algorithms for solving dynamic optimization problems;
- understands the main features of these algorithms, their strengths and weaknesses including their convergence properties;
- can implement them in an appropriate language;
- can model real-world decision problems into a DP or RL framework and solve moderately sized problems;
- has knowledge of the historical development of DP and RL and has an idea of possible future developments.

Course Content

This course is concerned with reinforcement learning and its origin dynamic programming. These are fields dealing with goal-directed decision making over time, such as finding your way in an unknown area, playing a game or pricing airline tickets.
We look at these areas from different angles:

- we deal with full-information "planning" problems, but also with partial-information "learning" problems
- we consider different algorithms, some of which are guaranteed to find the best solution, but also heuristics
- we consider high-dimensional problems (such as games) and methods to solve them

- we look at small toy problems to understand algorithms and sharpen our intuition, but also bigger problems for which we learn how to implement algorithms (in python)
- we look at different types of applications, both from AI (search problems, games) and OR

Additional Information Teaching Methods

Lectures and practical work integrated

Method of Assessment

Programming exercises and final exam.
The 3 assignments each count for 10% and the exam for 70%.
The minimal passing grade for the exam is 5.0.

Literature

Slides and lecture notes

Additional Information Target Audience

mBA, mAI, mCS, mBa-D, mMath

Recommended background knowledge

Programming experience in python

E-Commerce Law

Course Code	R_E.commerc
Credits	6
Period	P3
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Law
Course Coordinator	P.B. Cardozo Solano LLM
Examiner	P.B. Cardozo Solano LLM
Teaching Staff	prof. mr. A.R. Lodder, P.B. Cardozo Solano LLM
Teaching method(s)	Study Group, Reading

Course Objective

This course aims to provide a general understanding of the European legal framework concerning e-commerce and platforms. The European Union regulations and directives related to e-commerce are a starting point in this course. Specific topics that will be addressed are:

1. The position of online intermediaries
2. Electronic contracting and information requirements
3. Platform power
4. Unfair commercial practices in e-commerce
5. E-Commerce entrepreneurship
6. Commercial communications
7. Online reviews
8. Online dispute resolution

Students will learn to find the main rules related to e-commerce and platforms, describe and apply these rules to cases, critically assess the rules and think creatively about how legal rules may be improved or technical solutions may solve issues.

Course Content

The European Union has a long history of investing in, encouraging, and developing e-commerce. During the past 20 years, the EU has established an elaborate and to a certain extent, coherent regulatory framework for the e-commerce sector. This course gives insight into the main European legal framework regarding e-commerce and platforms. We will discuss the main legal issues, such as the liability of intermediary service providers, electronic contracting, electronic identification, and online dispute resolution. The legal rules will be critically assessed from a

theoretical and practical point of view. Regarding the theoretical approach, critical assessments in legal literature are taken as a point of departure. A critical assessment from a practical point of view can be achieved by applying the rules to cases and identifying possible bottlenecks.

Additional Information Teaching Methods

The course will be taught in seven two-hour lectures and three two-hour tutorials. The tutorials are interactive and you will need to prepare. During the first lecture, students will be introduced to the course and to their fellow students (the background of the students is always very interesting, many nationalities, with different backgrounds, for example legal, technical etc.).

Method of Assessment

The mid-term assignment is an individual and it is worth 30% of the final grade. A final assessment worth 70% of your overall mark is required. You will be asked to work in teams on this submission. As there is always discussion about possible answers and positions, it helps to develop your arguments if you work in teams.

Literature

Mandatory and recommended literature for each lecture and tutorial is available via Canvas.

Additional Information Target Audience

Apart from regular VU law students, the course is also available for:

- Students from other universities and faculties
- Exchange students
- Contractors (students who pay for one course)

Courses from a master at the faculty can only be taken as a secondary course if you have a diploma that gives access to the relevant master/specialization and if you are enrolled in a master's.

Additional Information

At the end of this course students:

- Are able to identify the relevant EU laws for e-commerce and platforms
- Understand (in general) the implications of EU Directives and Regulations and the role of the CJEU
- Understand what is meant by the key concepts within the legal framework (such as: information society service, trader, and consumer)
- Understand the reforms in platform regulation when performing commercial practices.
- Can reflect on the legal rules that are discussed during this course, more in particular if they solve the problems related to e-commerce and what (new) problems may arise as a result of the rules
- Can suggest legal or practical solutions regarding the practical or legal problems related to e-commerce and platforms.

Energy-efficient Edge Computing

Course Code	XMU_0056
Credits	6
Period	P5
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	J.V.A. Tielen MA
Examiner	
Teaching Staff	
Teaching method(s)	

Course Objective

This is an UvA course. Please follow the link for the course description.

Course Content

https://studiegids.uva.nl/xmlpages/page/2025-2026-en/search-course/course/130405

Additional Information Teaching Methods

-

Method of Assessment

-

Custom Course Registration

This course is offered by the UvA. For more information contactFNWI Education Service Centre.

Enrolment via https://glass.uva.nl is required.

Entrepreneurship in Analytics and AI

Course Code	XM_0090
Credits	6
Period	P4
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. W. Stam
Examiner	prof. dr. W. Stam
Teaching Staff	prof. dr. W. Stam, H.V. Nguyen, drs. G.J. Beekman MA, P. Afonso Branco Ramos Pinto
Teaching method(s)	Seminar, Written partial exam, Lecture

Course Objective

Academic and research skills: Students are able to generate a promising new business idea in the field of AI & Analytics and develop a prototype that validates the proposed customer needs and market potential.

Bridging theory and practice: Students understand the core theories and concepts in the field of entrepreneurship, as reflected by their ability to analyze, compare, and apply these theories and concepts to analyze and solve the key management challenges faced by real-world entrepreneurs.

Entrepreneurial soft skills: Students are able to effectively work in a team for generating and developing a new venture idea and can pitch the business concept to external stakeholders in a professional, engaging manner.

Course Content

Entrepreneurship in Analytics and AI is an engaging course for students who like to learn about entrepreneurship, its role and importance in our society, and the process by which entrepreneurs transform new ideas into successful AI-driven business ventures.

In this course, students will learn the ropes of this emerging field via three educational pillars:

1. Students will learn to understand and apply theories from economics, sociology, and psychology to study key topics in entrepreneurship, including innovation, new venture creation, entrepreneurial competences, entrepreneurial finance, scaling and growth of new ventures, corporate entrepreneurship, and entrepreneurial ecosystems. Theoretical understanding of these subjects is applied to real world cases focusing on key management challenges faced by entrepreneurs and how to creatively overcome these challenges.
2. Students will familiarize with a set of entrepreneurial soft skills, which they will put into practice when they will be asked to approach different stakeholders to further their team projects to gain commitments or to obtain important information. For this purpose, the students will receive training in generating initial ideas for opportunities as well as networking and network utilization.
3. Students will be introduced to the actions and strategies that promote the process of new venture creation. For this purpose, the students, supported by the lecturers and coaching, will write (and pitch) a Student Design Sprint report (including prototype) around an innovative new venture opportunity in the field of Analytics and/or AI that they deem interesting for entrance into the market.

Additional Information Teaching Methods

During this course, students will attend both (guest) lectures and tutorials. The lectures explain relevant concepts, theories, and frameworks in the field of entrepreneurship and illustrate their usefulness using various practical examples. The tutorials are designed to facilitate students in making progress with their team projects by providing appropriate guidance and feedback.

Method of Assessment

The final grade for this course is based on a written individual exam, several individual pass/fail assignments, and a Student Design Sprint (SDS) project assignment.

The written exam and pass/fail assignments are individual assessments whereas the SDS assignment is a team assessment. To pass the course, the exam and SDS assignment should be graded as sufficient. Both forms of assessment have one option for re-sit in case students received an insufficient score.

Literature

The literature for this course constitutes a selection of academic articles and case studies.

Evolutionary Computing

Course Code	X_400111
Credits	6
Period	P1
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. A.E. Eiben
Examiner	prof. dr. A.E. Eiben
Teaching Staff	prof. dr. A.E. Eiben, mr. J.M. di Matteo, dr. K. da Silva Miras de Araujo
Teaching method(s)	Seminar, Lecture

Course Objective

This course has a threefold objective:

1. (Knowledge & Understanding) To learn about computational methods based on Darwinian principles of evolution.
2. (Applying knowledge and understanding, Communication) To use such methods as problem solvers and as simulation tools; to determine adequate algorithm setups depending on the problem at hand.
3. (Applying knowledge and understanding, Lifelong learning skills) To gain hands-on experience in performing computational experiments with evolutionary algorithms.

Course Content

This course is about constructing, applying and studying algorithms based on the Darwinian evolution theory. Driven by selection (survival of the fittest, mating of the fittest) and randomised reproduction (mutation, recombination), an evolutionary process is being emulated and solutions for a given problem are being "bred". During this course, various flavours within evolutionary computing are treated, including genetic algorithms, evolution strategies, evolutionary programming, genetic programming, differential evolution, particle swarm optimisation. Applications in optimisation, constraint handling, machine learning, and robotics are discussed. Specific subjects handled include: genetic structures (representations), selection techniques, sexual and asexual reproduction operators, (self-)adaptivity and methodological aspects, such as algorithm design & tuning, and performance assessment. Special attention is paid to the field of evolutionary robotics where the bodies (morphology, hardware) and the brains (controller, software) of robots are evolved simultaneously. Hands-on-experience with evolutionary algorithms is gained through a compulsory programming assignment.

Additional Information Teaching Methods

Oral lectures and compulsory Python programming assignment (in teams of 3 to 5). A limited number of highly motivated students can replace the programming assignment by a special research assignment under the personal supervision of the lecturer(s). These research projects aim at publications with the students as first authors of the paper.

Method of Assessment

Written exam and programming assignment (in teams of 3 to 5). To pass the course as a whole, you must pass both the exam and the programming assignment. For the final grade they will count with the same weight:

Grade = 0.5 x Exam_Grade + 0.5 x Assignment_Grade

Note that:

- The exam is multiple choice, taken on the campus in an electronic form. There is a resit in case you cannot attend or fail.
- The programming assignment is graded based on your code, the quality of the solution your algorithm finds, and a short report. **No resit is possible for the programming assignment.**
- If you pass one component (Exam or Assignment), but fail the other, then you can keep the grade of the passed component for next year or later. Then you can pass the course by getting a good grade (> 5.5) for the missing component and using the old grade in the averaging formula.

Literature

The course is based on the textbook ***A.E. Eiben and J.E. Smith, Introduction to Evolutionary Computing, Springer, 2015, 2nd edition, ISBN 978-3-662-44873-1***. The lectures and the book chapters are synchronised to a great extent. In addition, a number of carefully selected papers to provide extra information on specific topics are available via Canvas.

Additional Information Target Audience

MSc Artificial Intelligence
MSc Econometrics and Operations Research
MSc Finance
MSc Business Analytics
MSc Computer Science
MSc Computer Security
MSc Bioinformatics and Systems Biology

Additional Information

Attending the lectures in real life is not mandatory, but highly recommended. Active participation in the team work for the programming assignment is a must!

Explanation Canvas

The complete schedule of the course, organised by the chapters of the textbook, will be made available at the beginning of the course. Students are assumed to consult the course schedule to keep up with the progress, as Canvas rubrics are not used. Video recordings of the lectures made during the COVID period are still available on Canvas for self-study.

Recommended background knowledge

Python programming skills are necessary to do the practical assignment. If you cannot program, you cannot complete this course.

Experimental Design and Data Analysis

Course Code	X_405078
Credits	6
Period	P4
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. E.N. Belitser
Examiner	dr. E.N. Belitser
Teaching Staff	dr. E.N. Belitser
Teaching method(s)	Practical, Lecture

Course Objective

In this course, students will become familiar with the most commonly used experimental designs, regression

models, some nonparametric tests and bootstrap methods. By the end of the course, students will be able to:

- Identify and describe some of the most widely used statistical models,
- Understand the basic theory underlying these models,
- Apply these models appropriately in relevant contexts.

Course Content

The EDDA course introduces a range of widely used statistical models and techniques, including paired and independent t-tests, non-parametric rank-based tests, bootstrap and permutation methods, analysis of variance (ANOVA), ANCOVA, lasso, linear regression models, generalized linear and nonlinear models, and survival analysis. Students will learn how to choose appropriate models for various scenarios, estimate model parameters using available data, and validate model fit. For regression models, proper study design is essential to draw sound conclusions; the course covers well-known experimental designs such as completely randomized and randomized block designs, along with their associated ANOVA techniques. A portion of the course focuses on non-parametric and resampling methods, such as the Wilcoxon test (one- and two-sample), Kolmogorov–Smirnov test, rank correlation tests, and both permutation and bootstrap tests. The methods will be illustrated using practical examples, with an emphasis on real datasets. The statistical software R will be used for data analysis throughout the course.

Additional Information Teaching Methods

Lectures, practical sessions.

Method of Assessment

Course assessment consists of assignments (33.3%) carried out in teamwork during the course and an individual digital test (66.7%) at the end. To pass the course, both components must meet minimum required thresholds. Re-examination policy: if the final result is insufficient, only the individual test may be retaken.

Literature

Literature will be communicated on Canvas.

Additional Information Target Audience

MSc AI, CS

Additional Information

Assignments are to be made using the statistical package R (http://www.r-project.org).

Explanation Canvas

All the information about the course will be available on Canvas.

Recommended background knowledge

- A basic course in statistics of the same level as Statistical Methods (X_401020) or Statistical Methods for AI (XB_0080) is mandatory. All the material and its prerequisites (basic mathematical and programming skills) for these courses will be assumed known in the lectures/assignments/exam of the course EDDA.
- A good background in probability, at least of the same level as in Statistical Methods (X_401020) or Statistical Methods for AI (XB_0080).
- Basic knowledge of the statistical software R and its application to data analysis.

Foundations of Cryptography

Course Code	XM_0174
Credits	6
Period	P2
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	S.A. Oechsner
Examiner	S.A. Oechsner

Teaching Staff	S.A. Oechsner
Teaching method(s)	Seminar, Lecture

Course Objective

Upon completion of this course:

- Students are familiar with the most fundamental cryptographic building blocks and the security guarantees they can(not) provide. (Knowledge and understanding)
- Students know the basic security models used in cryptography. (Knowledge and understanding)
- Students can reason about the security of basic cryptographic building blocks in these models. (Applying knowledge and understanding)

Course Content

Cryptography forms the foundation for the security of many digital systems around us. In this course, we will study how the basic cryptographic building blocks work, and why they work: What does it mean to be secure? How can security be formally defined, and how does one formally reason about security?

Covered topics include:

- the provable security mindset
- pseudorandom functions
- symmetric encryption
- message authentication codes
- collision-resistant hash functions
- public-key encryption
- digital signatures

Additional Information Teaching Methods

Lectures (4 hours per week) and exercise class (2 hours per week).

Method of Assessment

Written exam.

Literature

We will be using "The Joy of Cryptography" by Mike Rosulek as textbook. Available as free pdf at https://joyofcryptography.com.

Recommended background knowledge

Knowledge of basic discrete mathematics (simple combinatorics, discrete probability, modular arithmetic). Prior exposure to abstraction in the context of computer science is helpful, e.g. in the form of automata theory as taught at the basic CS bachelor level.

Functional Programming

Course Code	XMU_0062
Credits	6
Period	P4
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	J.V.A. Tielen MA
Examiner	
Teaching Staff	
Teaching method(s)	

Course Objective

This is an UvA course. Please follow the link for the course description.

Course Content

https://studiegids.uva.nl/xmlpages/page/2025-2026-en/search-course/course/130042

Additional Information Teaching Methods

-

Method of Assessment

-

Custom Course Registration

This course is offered by the UvA. For more information contactFNWI Education Service Centre.

Enrolment via https://glass.uva.nl is required.

Fundamentals of Adaptive Software

Course Code	XM_0128
Credits	6
Period	P2
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. I.G. Gerostathopoulos
Examiner	dr. I.G. Gerostathopoulos
Teaching Staff	dr. I.G. Gerostathopoulos
Teaching method(s)	Seminar, Written partial exam, Lecture

Course Objective

Upon completion of the course, students will be able to:

- understand and explain the principles and terminology of self-adaptation (what it is, why it is useful) and its uses in different domains (Knowledge and understanding);
- describe adaptation loops based on the MAPE-K model - by Monitoring its execution context, Analyzing the gathered information to reveal adaptation needs, Planning adaptation strategies, and Executing the identified adaptation strategies. All the steps in the adaptation loop may share a common Knowledge base (Knowledge and understanding);
- understand and apply adaptation patterns (Knowledge and understanding) (Applying knowledge and understanding);
- design and develop an adaptive software able to deal with uncertainties and trade-offs among various quality attributes of the software at runtime based on MAPE-K (Applying knowledge and understanding);
- evaluate, categorize, and critically discuss the benefits of the adaptive software and its development effort (Making judgements) (Communication skills).

Course Content

Today’s software systems run in dynamic environments and are expected to operate continuously even in presence of internal faults, external variabilities, and uncertainties. They are required to adapt themselves to the current execution environment without (or with minimum) human intervention while ensuring a certain level of quality for the provided functionality.

This course will focus on the fundamentals of adaptive software systems and cover both theoretical and practical aspects. Adaptivity is a central and timely research topic in software and systems engineering and expected to play an even more important role in the future of interconnected devices and humans. Examples of application domains where adaptivity is successfully applied include cloud/edge computing, Internet of Things, Cyber Physical Systems, multimedia, robotics, healthcare. For example, in cloud computing, adaptivity may be used to achieve elasticity and flexibility in runtime resource management.

Students will learn about the main steps of adaptivity based on the MAPE-K loop: Monitoring, Analysis, Planning,

and Executing using a Knowledge base. They will be introduced to the various facets of adaptivity: self-optimization, self-configuration, self-organization, self-protection, and self-healing. Students will learn about uncertainties and variabilities addressed through adaptivity. Students will learn about reactive and proactive approaches to adaptivity.

Various adaptation strategies will be presented in different application domains. Finally, students will learn examples of design patterns for adaptivity.

Students will apply their knowledge in building (extending) existing self-adaptive software systems and perform their own research with them.

In particular, they will use their prototypes as a vehicle to apply relevant design patterns and identify, measure, and assess the benefits of adaptation, as well as the development and maintenance effort of adaptation.

Additional Information Teaching Methods

The course entails 3 live sessions per week: 2 lectures and one seminar. Attendance is mandatory.

The students are expected to participate in the live sessions, work on a group project and work on individual assignments.

Method of Assessment

- Final exam (individual, 30%) and final project (group-based, 70%).
- Resit options: retake of the final exam, re-submission of the final project.
- The assessment of the final project will include presentations, a final report, and the developed software (and its documentation).
- To pass the course, each student needs to obtain a sufficient grade both in the final exam (>=55%) and in the group project (i.e. average grade on the group project reports >=55%).

Literature

The course material will be handed out by the lecturers and distributed online (Canvas).

An example is: Weyns, Danny. Software engineering of self-adaptive systems: an organized tour and future challenges. Chapter in Handbook of Software Engineering. Springer. 2017.

Additional Information Target Audience

MSc Computer Science
MSc Information Sciences
MSc Computer Security

Custom Course Registration

Further information for this course will be made available online (Canvas). All students must be enrolled in the course Canvas community.

Group/project enrollment takes place in Canvas.

Explanation Canvas

The course material, the project related information, and exam information will be made available on Canvas.

Recommended background knowledge

Software Engineering basics.
Object-Oriented Programming basics.
UML - Unified Modeling Language basics.

Green Lab

Course Code	X_418158
Credits	6
Period	P1
Course Level	500
Language Of Tuition	English

Faculty	Faculty of Science
Course Coordinator	dr. I. Malavolta
Examiner	dr. I. Malavolta
Teaching Staff	dr. I. Malavolta, prof. dr. P. Lago, dr. V. Stoico
Teaching method(s)	Practical, Lecture

Course Objective

Upon completion of this course, students will:

- Learn the principles of Empirical Software Engineering. (Knowledge and understanding)
- Be able to operate in a lab environment and build a successful experiment for software energy consumption. (Applying knowledge and understanding) (Making judgements) (Lifelong learning skills)
- Become familiar with relevant research problems in the field of Green Software Engineering. (Knowledge and understanding)
- Understand and measure the impact of software over energy consumption. (Applying knowledge and understanding) (Making judgements)

Course Content

Students will work in teams to perform experiments on software energy consumption in a controlled environment. Students will have to carry out all the phases of empirical experimentation, from experiment design to execution, data analysis, and reporting. Students will be provided with examples of experiments coming from state-of-the-art literature, but they will have to choose by themselves the experimental subjects and hypotheses to test. During the lab sessions, students will be assisted for technical operation of the lab equipment as regards measurement and data gathering. Students will also receive the required training for data analysis and visualization using specialized software (e.g., R and RStudio).

Additional Information Teaching Methods

Lectures (H). Lab sessions (pra).

Method of Assessment

Students will be assessed via (1) a team project and (2) an individual presentation.

The team project will be carried out throughout the whole course by groups of 5 students; each part of the project will be discussed during the lectures, started during the labs, and completed as homework so that students will likely be on track within the course schedule.

The result of the team project is about planning, designing, conducting, and reporting a successful experiment for software energy efficiency in a specific technological context, such as mobile applications, robotics, and microservices. In this way, students will put into practice the skills and techniques that they have learned during the lectures and develop their practical insights by applying them to real software. The experiment will be evaluated by the instructor starting from a written report describing the main aspects of the experiment and according to a shared assessment rubric (presented in the first lecture of the course).

Presentations will be prepared as group work and they will be about the main points of the team project. Each student will present individually a part of the presentation, which will be graded as either “pass” or “fail”.

All assessment components (e.g., team project and individual presentations) have a resit opportunity.

Literature

- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., and Wesslén, A. Experimentation in software engineering. Springer, 2012.
- I. Malavolta, V. Stoico, P. Lago. Ten Years of Teaching Empirical Software Engineering in the context of Energy-efficient Software, 2024. Online: https://arxiv.org/abs/2407.05689
- Material distributed on Canvas.

Additional Information Target Audience

MSc Computer Science
MSc Computer Security

Recommended background knowledge

Python programming and shell scripting.
Basic statistical analysis techniques (descriptive statistics and most common tests).
Academic writing.

High Performance Computing and Big Data

Course Code	XMU_40013
Credits	6
Period	P3
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	J.V.A. Tielen MA
Examiner	
Teaching Staff	
Teaching method(s)	

Course Objective

This is an UvA course. Please follow the link for the course description.

Course Content

https://studiegids.uva.nl/xmlpages/page/2025-2026/zoek-vak/vak/130398

Additional Information Teaching Methods

-

Method of Assessment

-

Custom Course Registration

This course is offered by the UvA. For more information contactFNWI Education Service Centre.

Enrolment via https://glass.uva.nl is required.

History of Digital Cultures

Course Code	XM_0134
Credits	6
Period	P3
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. D.J. Beckers
Examiner	dr. D.J. Beckers
Teaching Staff	dr. D.J. Beckers
Teaching method(s)	Seminar, Lecture

Course Objective

Students acquire a firm historical knowledge of computing in its cultural context. They will recognize the various strands in theories of “the information society”, of “digital culture”, and of "algorithmic thinking", and are aware of the strengths and limitations of such theories. (Knowledge and understanding) (Applying knowledge and understanding) (Making judgements)

Students gain experience in teamwork in a project. (Communication) (Lifelong learning skills)

Through the experience of thinking along the lines of a different discipline (i.c. history), students of computer science get the habit of making their own discipline the subject of reflection in a systematic way. (Making judgements) (Communication) (Lifelong learning skills)

Course Content

History of Digital Cultures is a hands-on reconnaissance tour through the history of the information society, from computers and software to the internet, and the present ubiquity of digital cultures.

The computer may be considered an information machine, an automatic calculator, or control technology. Three traditions build up the background for the advent of the stored program computer around 1950: data processing, process control, and scientific computation. The advent of the modern computer is treated here in its cultural context. An historical view on software is a major topic in this course. With the shaping of the internet, computers have sunk into the background. Computing and algorithms have not.

Treating the world as "data" has become second nature. History will help us recognise these aspects of today's culture. Throughout history, speculations on the societal impact of computing have accompanied this technology. History of Digital Cultures addresses computing in society and on a more abstract level theories of “the information society”, of “digital culture”, and of "algorithmic thinking"

History is concrete and local. The projects, building up the course, will be concerted efforts to explore local or national developments, possibly concerning The Netherlands, like Dutch computer- and software pioneers, contributions to software engineering, and more recent specialities like the Dutch role in the internet, "De Digitale Stad" (DDS), web design, gaming, and digital art.

The course is an intensive, engaging, and fully committing four-week project in teamwork. We expect students to be working full-time on the project. To actively participate in their own team (on site), but also reacting to the presentations of the other teams during the weekly presentations. The teaching team will be available "24/7" to assist teams and provide feedback.

Teams of 4 students choose their topic; the choice is open but we present options (with literature) on the Canvas site which you may use for inspiration.

Additional Information Teaching Methods

Seminars / working independently on project / feedback meetings / lectures / excursion.

Each lecture (except the first one) will finish with a short individual reflection on what the lecture contributed to the own research project. Every Friday there is a seminar where students present their progress to the teaching team and their fellow students. By providing and receiving (peer) feedback during these meetings, we also get inspiration from each other. Attendance of the Friday seminars is mandatory. During the weeks, students will be working independently on their group projects, but several feedback meetings are scheduled. During these meetings, groups report on their projects to TAs, receive feedback and may pose questions. One excursion at least is scheduled, participation is on a voluntary basis.

Method of Assessment

Project assignments, presentations, project report.

The weekly presentations and draft reports are graded tentatively. Most important is the feedback the teaching team provides.

The final grade is calculated on the basis of:

1. Individual reflections to the lectures (pass/fail; two fails are allowed)
2. End presentation by the group (20%)
3. End report (50%)
4. Individual oral exam on the content of literature, lectures and project (30%) must be passed with 5.5 minimum

Literature

We start by reading and collectively discussing David Gugerli's "How the world got into the computer". More material will be announced on Canvas.

Additional Information Target Audience

MSc Computer Science

ICT for the Global South

Course Code	XM_0142
Credits	6
Period	P5
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. K. Aslam
Examiner	dr. K. Aslam
Teaching Staff	dr. J. Gordijn, dr. K. Aslam
Teaching method(s)	Seminar, Lecture

Course Objective

At the end of the course, students will be able to:

- Gain a comprehensive understanding of the digital divide, challenges and opportunities in the Global South and the role of ICT in addressing them. (Knowledge and Understanding)
- Apply technical skills and innovative approaches to develop innovative ICT solutions to meet specific needs in the Global South. (Applying Knowledge and Understanding)
- Evaluate the impact of ICT policy and governance on socio-economic development and how it contributes(d) to digital divide by considering ethical, cultural, and sustainability factors. (Making Judgements)
- Effectively communicate project design and outcomes both orally and in academic reports. (Communication)
- Develop the capacity for self-directed learning about emerging technologies and their potential applications in ICT for the Global South. (Lifelong Learning Skills)

Course Content

This course explores the role of Information and Communication Technologies (ICTs) in addressing socio-economic challenges faced by developing countries, commonly referred to as the Global South. It provides a comprehensive understanding of how ICTs can be harnessed to foster development, improve governance, enhance education, healthcare, and empower communities.

The lectures (in the form of interactive sessions) will explore how technological innovations can meet specific local needs through sustainable and culturally relevant solutions. Students will engage in hands-on projects to apply their technical skills and innovative thinking to develop real-world ICT solutions tailored to the nuanced demands of the Global South.

Students will be guided during the course to develop a reflective practice that includes empathy and ethical considerations, essential for developing ICT solutions for low resource environments. The course focuses that ICTs are powerful enablers for achieving many of the United Nations Sustainable Development Goals (SDGs), teaching students how to leverage technology to address some of the world's most pressing challenges.

Additional Information Teaching Methods

This lectures will be delivered in an interactive seminar-style format. Prior to each class, students are expected to engage with assigned readings and multimedia resources to build a foundational knowledge base. During the lecture, students will discuss main insights and reflect on their own viewpoint in relevance to the pre-class materials.

Method of Assessment

During the course students will work in a group of 3-5 (depending on the total number of students) to derive a solution for a particular problem in the context of Global South. The projects will be research-based and along the way students will need to submit four assignments in total:

Assignment #1: group (a written report of overall description of project stating goal and research questions) - 15%
Assignment#2: group (a written report of detailed description of methodology and plan to conduct the study) - 15%
Assignment#3: group - (a written report of results achieved, discussion of results in the specific context for which study is conducted and future outlook of the project) 40%
Assignment#4: individual oral presentations - 20%
Attendance: 10%

Each of the assignments should be at least graded with a 5.5 or higher and the total marks should be 6 or higher for a student to pass the course. One resubmission is allowed for Assignment#1 and Assignment#2 in case of achieved grade below 5.5. There is no resubmission for Assignment#3 and Assignment#4.

There will be no final exam.

Literature

Relevant literature will be made available on Canvas.

Additional Information Target Audience

MSc AI, MSc CS, MSc IS

Recommended background knowledge

We expect students to have at least a basic knowledge of programming and how ICT projects are conducted. Familiarity with mixed-methods research or design science will be a plus, but not mandatory.

Inclusive ICT

Course Code	XM_0177
Credits	6
Period	P6
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. K. Aslam
Examiner	dr. K. Aslam
Teaching Staff	dr. K. Aslam, dr. J. Gordijn
Teaching method(s)	Study Group, Lecture

Course Objective

At the end of the course, students will be able to:

1. Develop an understanding of the socio-economic characteristics of marginalized and minority populations in the global North and the disparities in ICT access and usage. (Knowledge and Understanding)
2. Elicit the specific requirements and limitations of marginalized populations and then propose tailored ICT solutions for them. (Applying Knowledge and Understanding)
3. Analyze the potential impact and ethical implications of the designed ICT solutions on the target populations. (Making Judgements)
4. Effectively communicate the outcomes of ICT needs assessments and solution proposals both in written reports and oral presentations. (Communication)
5. Engage in continuous learning about the evolving needs of marginalized populations in relation to ICT applications and maintaining an adaptive approach to problem-solving in these cases. (Lifelong Learning Skills)

Course Content

This course will provide students with an understanding of the global North's socio-economic landscape and the marginalized and minority populations within it (e.g., differently-abled people, migrants and refugees, elderly population, ethnic minorities etc.) exploring disparities in ICT access. The course objectives closely align with several of the United Nations Sustainable Development Goals (SDGs) such as Reduced Inequalities (SDG 10), Sustainable Cities and Communities (SDG 11), and Peace, Justice, and Strong Institutions (SDG 16).

We will discuss methods for assessing ICT needs for these populations through case studies and community engagement activities. Students will learn about designing inclusive ICT solutions also by working together in groups where they will propose and present a tailored ICT application. The course will also incorporate discussions on the effectiveness of public-private partnerships and policy implementations in fostering ICT accessibility in the Global North.

Additional Information Teaching Methods

The lectures will be delivered in an interactive seminar-style format. Prior to each class, students are expected to engage with assigned readings and multimedia resources to build a foundational knowledge base. During the lecture, students will also reflect on their own viewpoint in relevance to the material discuss in class.

Method of Assessment

Two group assignments: 60%
Four pass/fail homeworks (individual, may also include delivering recorded presentation): 30%
Attendance: 10%

Each of the assignments should be at least graded with a 5.5 or higher and the total marks should be 6 or higher for a student to pass the course.

Literature

Literature will be made available on Canvas.

Additional Information Target Audience

MSc CS, MSc AI, MSc IS

Information Theoretic Learning

Course Code	XMU_0064
Credits	6
Period	P2
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	J.V.A. Tielen MA
Examiner	
Teaching Staff	
Teaching method(s)	

Course Objective

This is an UvA course. Please follow the link for the course description.

Course Content

https://studiegids.uva.nl/xmlpages/page/2025-2026/zoek-vak/vak/130408

Additional Information Teaching Methods

-

Method of Assessment

-

Custom Course Registration

This course is offered by the UvA. For more information contactFNWI Education Service Centre.

Enrolment via https://glass.uva.nl is required.

Introduction to Computational Science

Course Code	XMU_418111
Credits	6
Period	P1
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	J.V.A. Tielen MA
Examiner	
Teaching Staff	

Teaching method(s)

Course Objective

This is an UvA course. Please follow the link for the course description.

Course Content

https://studiegids.uva.nl/xmlpages/page/2025-2026/zoek-vak/vak/130584

Additional Information Teaching Methods

-

Method of Assessment

-

Custom Course Registration

This course is offered by the UvA. For more information contactFNWI Education Service Centre.

Enrolment via https://glass.uva.nl is required.

Knowledge Organization

Course Code	X_405065
Credits	6
Period	P1
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. R.M. Siebes
Examiner	dr. R.M. Siebes
Teaching Staff	dr. V. de Boer, dr. R.M. Siebes, T. Kuhn
Teaching method(s)	Seminar

Course Objective

At the end of this course, students will:

- have gained high-level knowledge of the potentials and problems with formal knowledge organization (knowledge and understanding);
- have gained high-level knowledge of the possible solutions and applications for formal knowledge organization (knowledge and understanding);
- be able to understand concrete knowledge organization methods and practically apply them (knowledge and understanding, apply knowledge and understanding);
- be able to communicate own thoughts and reflections on the topic of knowledge organization (communication)

Course Content

This course covers the general principles and methods that form the foundation of information organization and knowledge-intensive processes, as well as the contexts in which they can be applied and the interaction with users. The lecture topics include knowledge modeling, ontologies, logic, controlled natural language, Semantic Web and Linked Data, as well as knowledge maintenance and evaluation, in addition to guest lectures on specific applications and domains.

Additional Information Teaching Methods

Lectures are in a hybrid setup and can be followed both in the lecture room and online, and will be video-recorded. These recordings are made available through Canvas afterwards.

Method of Assessment

Final written exam (75%), final assignment (25%). All parts are individual (no group work). To pass the course students need a passing grade (5.5 or more) both for the exam and for the final assignment. There will be a resit for the written exam and for the final assignment. For the final assignment, the maximum resit grade is 6.0.

Literature

Papers to read will be announced through Canvas.

Additional Information Target Audience

MSc Artificial Intelligence
MSc Computer Science
MSc Information Sciences

Additional Information

The course has a maximum enrollment limit of 120. The students of the Master Information Sciences, for whom it is compulsory, have priority over the students that take this course as an elective or a constrained choice course.

The students that take this course as an elective or a constrained choice course run the risk of not being able to participate, even if they have registered on time, if the maximum enrollment limit has been reached.

The students of the Master Information Sciences, for whom it is compulsory, run the risk of being initially refused if the enrollment limit has been reached at the time of registration. However this will be adjusted manually after registration has been closed so that all the students of the Master Information Sciences will be able to participate.

Large Research Project Computer Science

Course Code	XM_0130
Credits	12
Period	Ac. Year (September)
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. ing. T. Kielmann
Examiner	dr. ing. T. Kielmann
Teaching Staff	dr. ing. T. Kielmann
Teaching method(s)	

Course Objective

The objectives of the course are:

- *Applying Knowledge and Understanding:* After completing the Research Project Computer Science, the student will be able to execute a small computer science project in the context of ongoing research.
- *Making judgements*: The student will be able to evaluate their own approach and results w.r.t. novelty and efficacy.
- *Communication*: The student will be able to communicate their results and findings in a scientific report.
- *Lifelong learning skills*: The student will be trained in applying their existing knowledge and skills in the context of ongoing research.

Course Content

The aim of the *Large Research Project Computer Science* is to give very motivated students the opportunity to work on a challenging, research-oriented project, with a strong focus on serious computer science conceptual and/or practical work.

The content of the project is to be negotiated individually with a VU or UvA Computer Science lecturer, who will supervise and grade it. This must be done before the project starts. In principle, only a few students (the ones that submit a convincing project proposal) will be allowed to take this course. The project can be carried out as a ‘duo project’, in which case the proposal should indicate the individual contribution of each of the two students.

The project is supposed to apply state-of-the-art concepts, methodologies and/or technology and requires an

original contribution by the student/s. It must be challenging, and allow a clear assessment of the student's own contribution.

At the end of the project, the student submits a written report to the lecturer, describing the work, the lessons learned, and the insights gained from applying study contents in a research-oriented computer science project.

The report contains a description of the project, including the problem description, the student's solution, and an evaluation of the results. The report also contains a reflection on the novelty of the achieved research results, a comparison with related work, and what the student has learned during the project.

In the case of a duo project, the report should clearly explain the individual contribution of each student so as to personalise their contribution to the relevant parts.

Additional Information Teaching Methods

Guided project work.

Method of Assessment

The overall grade is based on two components: research work (70%) and the quality of the written report (30%).

Each component must be passed with a grade >= 5.5. The final grade is the weighted average of the components' grades.

The report will be evaluated by the supervisor and by a second reader. For grading, there is a special evaluation form on the Canvas community page of the MSc Computer Science.

Literature

To be distributed individually, by the project advisors.

Additional Information Target Audience

MSc Computer Science

Additional Information

The Large Research Project Computer Science is available for the last time in academic year 2025/2026, for those students who started their studies in 2024/2025 or earlier.

Custom Course Registration

For being accepted, students must contact their prospective advisor directly.

Recommended background knowledge

The student should have completed at least 42 credits of their Master programme, so that there is sufficient study content to be applied in the project.

Literature Study

Course Code	XM_0131
Credits	6
Period	Ac. Year (September)
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. P. Lago
Examiner	prof. dr. P. Lago
Teaching Staff	prof. dr. P. Lago
Teaching method(s)	Lecture

Course Objective

After taking this course, the students will be able to:

- Design a sound desk research based on scientific literature and identify the relevant literature related to the target research; or perform an in-depth study of a selected set of literature sources (Knowledge and Understanding; Applying Knowledge and Understanding; Making judgements; Communication).
- Criticize, analyze, and discuss scientific literature (Applying Knowledge and Understanding; Making judgements; Communication; Learning Skills).
- Reflect on the in-depth knowledge gained during the course on the selected literature study topic (Making judgements; Communication; Learning Skills).

Course Content

The course consists of carrying out a literature study on a topic chosen in agreement with the selected tutor.

The ‘theoretical preparation’ of the course consists of studying the provided material on literature study design, where the student learns how to go from a research question to a well-structured analysis of the literature. This step can be carried out by translating the research question into a sound search query, identifying adequate online literature search engines, and performing a motivated selection of the literature for further analysis; or alternatively by selecting, together with the tutor, a set of relevant literature and performing an in-depth study.

The actual literature study starts with the 'exploration phase' in which the student must identify a topic of interest or a course particularly appreciated. He or she will then contact the person in charge of the identified research area/course and discuss with him/her the possibility of carrying out a literature study under his/her supervision.

In general, the literature study will be either predominantly broad, leading to a literature survey on the selected theme, or deep, carving out the intricacies of a specific topic. The outcome of the literature study is a final report, which must include: the study design; an overview of selected literature; an analysis of the literature; a discussion, and conclusions. At the end of this phase, the student gives one final presentation to the research group of the tutor. This presentation includes research questions, study design, study execution, and discussion of the analysis.

Additional Information Teaching Methods

Individual assignment. Regular progress meetings with the tutor.

The course entails 1 kick-off live lecture by the course coordinator, which provides an overview of the course and addresses questions from the students. This lecture is typically provided once at the beginning of the academic year.

Method of Assessment

Final report (V). Final presentation (pre).

The grading criteria are:

- quality of study design (if applicable);
- the rationale for literature selection (if applicable);
- quality of results (scientific quality of the analysis, discussion of the findings, reflection in the drawn conclusions, clear answer to main research question);
- quality of writing (style, clarity, organization);
- correctness and completeness of references and citations;
- final presentation to the research group where he/she carries out the literature study (narrative style, balance between focus and coverage of contents, answering questions).

Each grading criterion, if applicable, is evaluated on the scale Excellent-Very_Good-Good-Satisfactory-Sufficient-Insufficient. The final grade is calculated as the average of all criteria.

Literature

The introductory material on literature study design is on Canvas.

Additional Information Target Audience

MSc Computer Science

Additional Information

The student should look for a tutor in the area he or she would like to carry out the literature study.

See online material (Course reading) for further information.

Tutors can be chosen among the staff members of the VU CS Department and the UvA Informatics Institute.

External tutors, e.g., experts in a specific field, are allowed/encouraged, in which case a VU or UvA internal tutor should be formally involved for registering the final grade.

Litereature Study is available for the last time in academic year 2025/2026, for those students who started their studies in 2024/2025 or earlier.

Custom Course Registration

Depending on their study plans, each student may schedule the course flexibly. It is, however, recommended in the second year of studies.

Explanation Canvas

FAQ are answered via Canvas. Potential tutors may propose online (on Canvas) topics for prospective students (if applicable).

It is advised to have the same tutor for the literature study course and the master project (if applicable): this allows the student to use the literature study to get acquainted with the topic of their thesis; however, the content of the literature study report, naturally, cannot be part of the thesis report.

Logical Verification

Course Code	XM_0167
Credits	6
Period	P1
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. K. Sojakova
Examiner	dr. K. Sojakova
Teaching Staff	dr. K. Sojakova
Teaching method(s)	Seminar, Lecture

Course Objective

The course introduces interactive theorem proving via the proof assistant Lean. We study its logical foundations and its applications to computer science and mathematics. The course's goals are that the students:

1. learn fundamental theory and techniques in interactive theorem proving; (Knowledge and understanding)
2. learn how to use logic as a precise language for modeling systems and stating properties about them; (Applying knowledge and understanding) (Making judgements)
3. familiarize themselves with selected areas in which proof assistants are successfully applied, such as functional programming, the semantics of programming languages, and mathematics; (Knowledge and understanding) (Applying knowledge and understanding)
4. develop practical skills which they can apply on a larger project (as a hobby, for an MSc or PhD, or in industry); (Applying knowledge and understanding) (Learning skills)
5. get to understand the domain well enough to start reading relevant scientific papers published at international conferences such as CPP and ITP or in the Journal of Automated Reasoning. (Applying knowledge and understanding) (Learning skills) (Communication skills)

Course Content

Proof assistants can be used to check the correctness of the specification of a program or the proof of a mathematical theorem. In the practical part of the course, we use the proof assistant Lean 4 to prove mathematical theorems in a precise, formal way, and see how to verify small programs.

We cover the basics of interactive theorem proving, functional programming (datatypes, recursive functions), logic programming (inductive predicates), and semantics of imperative programming languages (operational and denotational; Hoare triples). We discuss some mathematical applications.

Additional Information Teaching Methods

2 lectures per week (90 minutes each), 2 practical sesssion per week (90 minutes each)

Method of Assessment

The written exam counts for 75%, and the homework assignments count for 25% of the final grade. There will be four assignments, each corresponding to one of the major topics of the course. Assignments consist of a theoretical part and a programming part in Lean.

Literature

The course notes "The Hitchhiker's Guide to Logical Verification", by Baanen, Bentkamp, Blanchette, Hölzl, and Limperg, available as a PDF on the course's web site.

Additional Information Target Audience

MSc Computer Science

Recommended background knowledge

The main prerequisites for this course are an introduction course in logic and familiarity with equational programming or functional programming (e.g., Haskell, OCaml, Scala, Standard ML).

Machine Learning for the Quantified Self

Course Code	XM_40012
Credits	6
Period	P6
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. M. Hoogendoorn
Examiner	prof. dr. M. Hoogendoorn
Teaching Staff	V.G.M. Francois Lavet, prof. dr. M. Hoogendoorn, B. Borsboom
Teaching method(s)	Practical, Lecture

Course Objective

The main aims of this course are to make the student familiar with specific machine learning techniques for quantified self and sensory data. Both the theory as well as the practical side of matters will be addressed. More specifically, the following aims are distinguished:

1. Understanding the domain of the quantified self and sensory data
2. Knowing and being able to apply techniques for outlier detection on sensory data.
3. Knowing and being able to apply techniques for feature engineering on sensory data.
4. Knowing and being able to apply clustering techniques related to sensory data.
5. Understanding the theory of supervised learning and its implications.
6. Learning how to apply non-temporal machine learning techniques to sensory data.
7. Knowing and being able to apply temporal machine learning techniques to sensory data, including deep learning techniques.
8. Knowing and being able to apply reinforcement learning in the application of the quantified self.

The course if focused on the Dublin descriptors Knowledge and understanding (learning new relevant algorithm in this domain); Applying Knowledge and Understanding (since the aim is also to apply the algorithms in practice); Making judgements (which technique is appropriate, how to best apply the techniques for a specific case); Communication skills (how to report on your approach, choices and your results), and Learning skills (able to find new relevant techniques, assess their suitability, etc.).

Course Content

The quantified-self refers to large-scale data collection of a user's behavior and context via a range of sensory devices, including smart phones, smart watches, ambient sensors, etc. These measurements contain a wealth of information that can be extracted by means of machine learning techniques, for instance for the purpose of predictive modeling. In addition, machine learning techniques can be a driver for adaptive systems to support users in a personalized way based on the aforementioned measurements. The type of data does however require specialized machine learning techniques to fully exploit the information contained in the data. Examples of challenges include the temporal nature of the data, the variety in the type of data, the different granularity of various sensors, noise, etcetera.

In this course specific techniques to handle quantified self (or broader sensory data) will be treated. More in specific, it will address:

- Feature engineering (how do we come from raw data to usable features):
  * Removing noise from data
  * Handling missing data
  * Identifying (temporal) features
- Learning of user patterns:
  * Temporal machine learning approaches such as recurrent neural networks, time series analysis
  * Clustering approaches with dedicated distance metrics (including dynamic time warping)
- Adaptive feedback and support
  * Reinforcement learning
- Integration of the various components.

In addition, a number of real-life applications will be discussed. Next to lectures, there will be an extensive practical part, where students will learn to work with various algorithms work on a project they propose themselves.

Additional Information Teaching Methods

The course will be taught in four weeks. Both regular lectures (l, first two weeks) and practical sessions (p) will be organized. During the first two weeks the emphasis will be on the lectures and getting started with the practical assignment that ties closely to the content of the lectures. The third week will be working on the practical assignment fulltime. In the fourth week a mandatory final lecture will take place in which a selection of groups will present their project. The course ends with an exam.

Method of Assessment

Written exam (50%) and practical assignment (50%). For both parts, the grade needs to be sufficient to obtain a final grade. The practical assignment is done in groups of three. For the practical assignment, two intermediate versions of the report should be submitted for feedback that are given a pass/fail. Only in case of a pass for the two intermediate versions will a final grade be given for the practical assignment.

The written exam will be multiple choice and have a resit during the Summer period. On top, in case your assignment grade average is insufficient you are allowed to redo the assignments by improving it based on the feedback you received. The maximum grade for this resubmitted assignment is set to a 6.

Literature

Hoogendoorn, M. and Funk, B. Machine Learning for the Quantified Self - On the Art of Learning from Sensory Data, ISBN 978-3-319-66308-1, Springer Verlag 2018. The book can be downloaded free of charge through the UBVU, just follow the link to the book: http://dx.doi.org/10.1007/978-3-319-66308-1

Papers on state-of-the-art deep learning techniques.

Additional Information Target Audience

Master Artificial Intelligence, Master Business Analytics, Master Computer Science

Additional Information

Lecturer: Prof.dr. M. Hoogendoorn

Custom Course Registration

Registration for medical informatics (UvA) students: Please check the following document https://www.dropbox.com/s/a2kd7xacuxt5y8e/AI%20for%20Health%202022-2023.
and then register for secondary courses via https://vu.nl/en/student/information-about-registration-and-enrolment/in

Recommended background knowledge

Programming experience. Knowledge of basic machine learning algorithms.

Master Project Computer Science

Course Code	XM_0011
Credits	30

Period	Ac. Year (September)
Course Level	600
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	mr. S. Ud Din
Examiner	mr. S. Ud Din
Teaching Staff	mr. S. Ud Din
Teaching method(s)	Lecture

Course Objective

The Master Project Computer Science concludes the Master Computer Science programme.

By completing this project, a student will be able to:

- Identify a scientific and/or a research problem in a computer science-related field
- Formulate one or more research questions to guide the project
- Design a solution and research methodology to answer the research question(s) in a limited period of time, implement the solution and critically analyze the results
- Collaborate with supervisors and other students and communicate (both orally and in writing) about their progress, the results, and the lessons learned.

The Master project Computer Science covers all Dublin Descriptors:
(Knowledge and understanding) (Applying knowledge and understanding) (Making judgements) (Communication) (Lifelong learning skills).

Course Content

The Master Project is the programme component where scientific and professional skills are trained most extensively. The Master Project always involves an element of originality or creativity. A student will use all the knowledge acquired during the studies to perform a design task or to contribute to solving a scientific problem in the computer science field.

The project can take place either inside the Department of Computer Science of the VU or the Informatics Institute of the University of Amsterdam, or as an internship in a company. In the case of an internship, supervision will be in cooperation with a daily supervisor at the company. In any case, a detailed project proposal must be written and approved in advance by a staff member, who in this way agrees to act as the project's main supervisor. The proposal must include the problem statement, the research question(s), a short description of the proposed method, and a planning that also states the frequency the student will meet with the daily supervisor/main supervisor. The main supervisor needs to find a second reader for the thesis, who must also be a staff member.

Additional Information Teaching Methods

In most cases, the Master project is performed as an individual project, under close and regular supervision by the main supervisor and the daily supervisor. In exceptional and well-argued cases, students may execute projects in small groups, when the project lends itself for.

The project starts when the project proposal has been signed by both students and supervisors. The student plans regular meetings with the daily supervisor and the main supervisor to discuss the progress. Towards the end of the project, the student gives a presentation in the research group of the supervisor or at the company where the internship took place. The student submits their thesis which will be evaluated by the main supervisor and the second reader.

Method of Assessment

The final grade is determined by the project supervisor and a second reader, based on the quality of the research (3/6), the written thesis (2/6), and an oral presentation (1/6). Each component must be passed with a grade >= 5.5.

In the case of a group project, students have to write individual theses and give individual oral presentations, demonstrating their individual proficiency.

Entry Requirements

Students may enroll in the Master Project Computer Science only after accumulating a minimum of 84 EC from MSc Computer Science courses.

Additional Information Target Audience

MSc Computer Science

Additional Information

Useful information can be found on the Canvas page. There, you will also find links to the web pages of the research groups of the Department of Computer Science (VU) and the Informatics Institute (UvA), with options for master projects.

Multi-core Processor Systems

Course Code	XMU_0055
Credits	6
Period	P4
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	J.V.A. Tielen MA
Examiner	
Teaching Staff	
Teaching method(s)	

Course Objective

This is an UvA course. Please follow the link for the course description.

Course Content

https://studiegids.uva.nl/xmlpages/page/2025-2026-en/search-course/course/130404

Additional Information Teaching Methods

-

Method of Assessment

-

Custom Course Registration

This course is offered by the UvA. For more information contactFNWI Education Service Centre.

Enrolment via https://glass.uva.nl is required.

Network Security

Course Code	XM_0100
Credits	6
Period	P2
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	B. Chandrasekaran
Examiner	B. Chandrasekaran
Teaching Staff	B. Chandrasekaran, dr. M. Zhang
Teaching method(s)	Seminar, Written partial exam, Lecture

Course Objective

- Understand how the implicit notion of trust in the design of Internet protocols introduces numerous vulnerabilities across different protocols (K&U, LS).
- Understand how the web public-key infrastructure (PKI) ecosystem works, focussing both on the theory and (mal)practice of various security mechanisms and protocols (K&U, AK&U, LS, C).
- Analyze how attackers abuse the vulnerabilities (e.g., DNS spoofing, BGP route hijacks, and various amplification attacks), by studying several recent attacks (e.g., using DNS, BGP, and TCP protocols) (K&U, AK&U, MJ, C).
- Study the state-of-the-art mechanisms for mitigating today's attacks and review their effectiveness (K&U, AK&U, C).
- Learn how to inspect the design and development, and investigate the deployment of various security mechanisms or protocols (K&U, AK&U, LS).

The acronyms above refer to the Dublin Descriptors covered by each learning objective, specifically: Knowledge and Understanding (K&U), Applied Knowledge and Understanding (AK&U), Making Judgment (MJ), Learning Skills (LS), and Communication (C).

Course Content

This course focuses broadly on network and (the network side of) web security. We will focus on analyzing vulnerabilities in Internet protocols (e.g., DNS, BGP, and TCP), attacks (e.g., sniffing, spoofing, and hijacking) that exploit such vulnerabilities, and mechanisms to mitigate such attacks.

To learn the "state-of-the-art" attacks and mitigation strategies, the course will review and discuss several scientific papers published at top-tier venues.

Additional Information Teaching Methods

Lectures, quizzes, and mid-term and final exams.

Method of Assessment

Quizzes (30%), mid-term exam (30%), and final exam (40%).

To pass the course, you must satisfy the following criteria: (a) You must score at least 6.0 on average in the quizzes; (b) you must score at least 6.0 in the final exam, and (c) You must score 6.0 in total (i.e., with all three components taken together).

There is no resit opportunity for the quizzes or the mid-term exam. There will be one resit opportunity for the final exam.

This course does not allow students to carry over (partial) grades from last year.

[Fraud] We ensure that each student individually demonstrates their abilities through the requirement that the supervised mid-term and final exam be passed with an average of at least 6.0.

Please note that your grade will be registered as follows:
If you do not submit any review and do not present a paper, you will receive a NS.
In all other cases, you will receive either a numeric grade or incomplete (NVD).
If you did not meet the minimum grade requirements for the reviews, your grade will be NVD.
Otherwise, your final grade is your weighted average.
If you plan to drop the course, do not submit anything, as NVD is administratively considered a failing grade.

Literature

No set book. All materials will be made available during the course.

Additional Information Target Audience

MSc Computer Security
MSc Computer Science

Recommended background knowledge

A basic knowledge of network protocols, and a good grasp of probability as well as simple data structures and algorithms.

Parallel Programming Practical

Course Code	X_400162
Credits	6
Period	P3
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. F. Verdugo Rojano
Examiner	dr. F. Verdugo Rojano
Teaching Staff	mr. M. Cassar, drs. C. Verstoep, dr. F. Verdugo Rojano
Teaching method(s)	Lecture

Course Objective

At the end of the course, the students will:

- Be able to apply distributed parallel programming models to implement algorithms on large-scale computing systems. (Applying knowledge and understanding)
- Have hands-on experience in executing and testing parallel programs using a cluster computing environment. (Applying knowledge and understanding, making judgments)
- Be able to critically evaluate the performance of a distributed parallel program, identifying bottlenecks, and perform optimizations. (Applying knowledge and understanding, making judgments)
- Effectively communicate parallelization approaches and performance results by writing scientific reports. (Communication, lifelong learning skills)

Course Content

This course is a follow-up of "Programming Large-Scale Parallel Systems", which allows students to apply the knowledge studied in "Programming Large-Scale Parallel Systems" in a more practical, hands-on setting. Throughout the course, students will transform given sequential algorithms into parallel programs able to run on multiple compute nodes within a large-scale computing system. We will consider distributed programming models, including message-passing with MPI and the task-oriented model based on remote procedure calls included in the Julia programming language. The programs will be tested for correctness and benchmarked for performance on the DAS-5 cluster computer (see http://www.cs.vu.nl/das5). The performance of the program will be measured, analyzed and, whenever necessary, optimized. Profiling tools will be considered to identify bottlenecks and to guide optimization. A scientific report must be written for each problem discussing the parallelization approach, the implementation choices, and the outcomes of the performance analysis.

Additional Information Teaching Methods

- A kick-off meeting to introduce the course objectives and the practical assignments.
- After the kick-off meeting, students work independently towards the completion of the assignments.
- There are weekly meetings for Q&A and to guide students in different aspects of the work.

Method of Assessment

- (90%) Practical assignments that include practical programming work and written report.
- (10%) Individual oral exam, where students defend their work in a presentation about the assignments and by answering questions from the teachers.

Additional rules

- The grade of the practical assignments should be at least 5.5 to pass the course.
- The grade of the individual oral exam should be at least 5.5 to pass the course.
- There is no resit opportunity for the practical assignments nor the oral exam.

Literature

All course materials, including documentation and reference materials, will be made available through Canvas.

Additional Information Target Audience

MSc Computer Science
MSc Computer Security
MSc Computational Science

Recommended background knowledge

- We highly recommend having attended "Programming Large-Scale Parallel Systems" before joining "Parallel Programming Practical".
- The algorithms will be implemented in the Julia programming language. An introduction to parallel computing in Julia is given in the course "Programming Large-Scale Parallel Systems". Otherwise, students can study the lecture notes in https://www.francescverdugo.com/XM_40017/dev/.

Performance of Networked Systems

Course Code	X_405105
Credits	6
Period	P2
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. R.D. van der Mei
Examiner	prof. dr. R.D. van der Mei
Teaching Staff	prof. dr. R.D. van der Mei
Teaching method(s)	Lecture

Course Objective

Students will acquire basic knowledge of:

- Performance issues in communication systems, a complex interplay between communication networks (TCP/IP, UDP), application servers (HTTP, QUIC), services and clients
- Performance models for wireless and mobile network protocols (WiFi, GSM, GPRS, UMTS, HSDPA, 4G, 5G, 6G and mobile ad-hoc networks)
- Performance engineering principles and management of traffic in IP-based networks
- Performance modeling of streaming and elastic traffic
- Quantitative models for planning capacity in communication networks

Students will gain experience in modelling, engineering and planning performance of networked systems, and will learn how to tackle performance problems arising in today's information and communication systems.

Course Content

Over the past few decades, information and communication technology (ICT) has become ubiquitous and globally interconnected. As a consequence, our ICT systems are expected to process huge amounts of digital information, which puts a tremendous burden on our ICT infrastructure. At the same time, our modern society has become largely dependent on the well-functioning of our ICT systems: large-scale system failures and perceivable Quality of Service (QoS) degradation may completely disrupt our daily lives and have huge impact on our economy. These developments raise the need for understanding the nature and source of performance problems, and for finding effective ways to make our networks robust and reliable.

Motivated by this, the course will address performance-related challenges and solution methods in communication networks. To this end, we focus on performance modeling, traffic management, QoS provisioning and capacity planning for both wired and wireless/mobile communication networks (such as WiFi, 2G, 3G, 4G, 5G, 6G networks, and mobile ad-hoc networks). We address questions like:

- How do wired and wireless communication network technologies work?
- How can we model their performance?
- How can we manage and control traffic flows and provision QoS in IP-based networks?
- How can we deal with the omnipresence of uncertainty in virtually all aspects of communication networks?
- What techniques are available to model, analyze and optimize performance?

Additional Information Teaching Methods

Classroom lectures and a practical take-home homework assignment.

Method of Assessment

The assessment will be based on the homework assignment (total 20%) and a written exam (80%). Homework assignments can be made in groups of one or two students.

Literature

- A textbook on "High Performance Browser Networking" by Ilya Grigorik, O-Reilly, 2013.
- A reader on Stochastic Performance Modelling.
- Individual papers on specific topics.

Additional Information Target Audience

mBA, mCS, mPDCS, mCLS, mEctrie

Explanation Canvas

The relevant course information will be shared via CANVAS.

Recommended background knowledge

The students should have basic knowledge of computer networks.

Programming Large-Scale Parallel Systems

Course Code	XM_40017
Credits	6
Period	P1
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. F. Verdugo Rojano
Examiner	dr. F. Verdugo Rojano
Teaching Staff	dr. F. Verdugo Rojano, mr. M. Cassar
Teaching method(s)	Lecture

Course Objective

At the end of the course, students will be able to

- Understand the main principles of High Performance Computing (Knowledge and understanding)
- Design and implement efficient parallel algorithms on distributed-memory parallel computers (Applying knowledge and understanding)
- Make use of different distributed parallel programming models to implement these algorithms, including message passing with MPI and the task-based model based on remote procedure calls available in the Julia programming language (Applying knowledge and understanding).
- Analyze the performance of parallel applications and network topologies (Making judgements)
- Identify the main parallelization challenges in selected scientific computing applications (Knowledge and understanding)

Course Content

In this course, students learn how to design and implement computer programs that solve a problem in parallel on a distributed-memory computing system such as the supercomputers available in the top500 list (https://top500.org/). This is key in many real world applications to reduce execution time and to solve very large problems that cannot fit in a single compute node. You will learn how to design such parallel algorithms in order to minimize different types of performance overheads such as communication overhead, search overhead, and load imbalance. You will also learn how to analyze the theoretical and experimental performance of the resulting codes and the underlying computer network topology. Among all parallel programming models available for high performance computing, this course focuses on distributed-memory parallelization. We consider different distributed parallel programming models, including established techniques such as message-passing with MPI, but also modern approaches like the task-based model available in the Julia programming language. The course also discusses the application of these techniques to a set of real-world scientific computing applications, including partial-differential equations in solid and fluid dynamics, and n-body problems in astrophysics.

Additional Information Teaching Methods

- There are 2 class sessions per week. Part of the course follows the flipped classroom method. Pre-recordings, lecture notes, and exercises are made available in advance for self study, and follow up discussions take place in class meetings to deepen understanding.
- The course also includes the opportunity to run experiments on the DAS-5 computer cluster (http://www.cs.vu.nl/das5).

- Computer codes used in the course are implemented in the Julia programming language. Previous knowledge of Julia is not required as it will be learned as part of the course.

Method of Assessment

There is a final written exam (E) and a programming assignment (A). Their grades are combined as G = 0.1*A + W(E)*E, where

- W(E) = 1 for E < 7, and
- W(E) = 1-(E-7)/30 otherwise.

The grades E and A are at most 10 points each.

According to this formula the programming assignment is "quasi optional". There is little or no penalty for skipping the assignment in most of the cases. However, high-scoring students in the exam need the assignment for a perfect score.

Literature

Course material will be made available on Canvas. Part of the course is based on the lecture notes available at https://www.francescverdugo.com/XM_40017/dev/.

Additional Information Target Audience

Master Business Analytics
Master Computational Science
Master Computer Science
Master Computer Security

Additional Information

There is also a separate course "Parallel Programming Practical" (6 EC) in Period 3, the aim of which is to complement the contents of this course with further practical skills and experience.

Research Methods in Computer Science

Course Code	XM_0175
Credits	6
Period	P6
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. S. Lusini
Examiner	dr. S. Lusini
Teaching Staff	prof. dr. ir. A. Iosup, dr. S. Lusini
Teaching method(s)	Seminar, Lecture

Course Objective

Students who complete this course will be able to explain the relationship between research methodologies and research questions. Furthermore, they will be capable of articulating the relevance and applicability of specific research methods to various research problems. The detailed learning outcomes are reported below.

Upon successful completion of this course, students will be able to :

1. Independently find and use suitable primary and secondary sources on a topic of study; [Knowledge and Understanding, Applying Knowledge and Understanding]
2. Distinguish and compare a range of research methods used in Computer Science (e.g., Systems Engineering, Design, Experimental, Statistical, Qualitative, and Formal research methods and approaches); [Applying Knowledge and Understanding, Making Judgements]
3. Work in a team to collectively delineate a research topic and design a robust and methodologically justified research plan; [Communication, Making Judgements, Lifelong Learning skills]
4. Analyze and constructively criticize research methodology towards creating a personalized guide to research in the field; [Applying Knowledge and Understanding, Making Judgements]
5. Conduct a peer review of another (student group's) research proposal. [Applying Knowledge and

Understanding, Making Judgements, Communication, Lifelong Learning skills]

Course Content

Methods are the tools researchers use to conduct their work. Selecting and correctly applying an appropriate method is a crucial step in scientific research. This course introduces students to key research methods in Computer Science. Students will learn to engage with and compare multiple modes of academic inquiry, consider the importance of methodology in research design, and assess the influence of methods on research outcomes. Specifically, students will learn to formulate research questions and write research proposals. The lectures cover topics such as setting up research, understanding and avoiding plagiarism, and conducting scientific communication.

Additional Information Teaching Methods

Weekly lectures (2-3 per week)

Method of Assessment

Assessment is divided into individual and group components. For each component, students can earn grade points that will be added up as the overall grade.

Group Components (with individual contribution):

- **Research Proposal**: up to 6 grade points (Learning Objectives 1, 2, and 3). The group collaboratively defines a research proposal, with each member contributing individual components such as specific research questions and corresponding methodologies. Concretely, next to the context, problem statement, and main research question proposed together, each student will identify a specific research question, express and clarify it, and further explain the methodologically correct steps to address this research question. The group will then bundle all these elements into a well-structured Research Proposal, ready to be peer-assessed. The individual contribution to the Research Proposal will be tested in a controlled environment through a Final Exam (see below).

Individual Components:

- **Final Exam (oral)**: up to 2 grade points, focusing on the research questions and the related methodologies (Learning Objective 4). During this exam, students will explain their scientific research question (that they individually identified in the Research Proposal) and reflect on what makes it appropriate, and identify a methodology to address it and reflect on its suitability.
- **Peer Assessment:** up to 3 grade points (Learning Objective 5); each student will receive for assessment at most two Research Proposals from other groups, and provide for each of them an individual peer-assessment, structured and detailed as discussed in the course.

In case of a failing grade, the assignments can be resubmitted once. Because the partial scores add up to above 10, students have opportunities to excel beyond the explicit requirements and/or compensate for some basic mistakes. For the final grade, a maximum of 10 applies.

Literature

All required literature is provided via Canvas.

Research Project Computer Science

Course Code	XM_0129
Credits	6
Period	Ac. Year (September)
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. ing. T. Kielmann
Examiner	dr. ing. T. Kielmann
Teaching Staff	dr. ing. T. Kielmann
Teaching method(s)	

Course Objective

The objectives of the course are:

Applying Knowledge and Understanding: After completing the Research Project Computer Science, the student will be able to execute a small computer science project in the context of ongoing research.

Making judgements: The student will be able to evaluate their own approach and results w.r.t. novelty and efficacy.

Communication: The student will be able to communicate their results and findings in a scientific report.

Lifelong learning skills: The student will be trained in applying their existing knowledge and skills in the context of ongoing research.

Course Content

The aim of the *Research Project Computer Science* is to give very motivated students the opportunity to work on a challenging, research-oriented project, with a strong focus on serious computer science conceptual and/or practical work.

The content of the project is to be negotiated individually with a VU or UvA Computer Science lecturer, who will supervise and grade it. This must be done before the project starts. In principle, only few students (the ones that submit a convincing project proposal) will be allowed to take this course. The project can be carried out as a 'duo project', in which case the proposal should indicate the individual contribution of each of the two students.

The project is supposed to apply state-of-the-art concepts, methodologies and/or technology and requires an original contribution by the student(s). It must be challenging, and allow a clear assessment of the student's own contribution.

At the end of the project, the student submits a written report to the lecturer, describing the work, the lessons learned, and the insights gained from applying study contents in a research-oriented computer science project.

The report contains a description of the project, including the problem description, the student's solution, and an evaluation of the results. The report also contains a reflection on the novelty of the achieved research results, a comparison with related work, and what the student has learned during the project.

In case of a duo project, the report should clearly explain the individual contribution of each student so to personalise their contribution to the relevant parts.

Additional Information Teaching Methods

Guided project work.

Method of Assessment

The overall grade is based on two components: research work (70%) and the quality of the written report (30%). Each component must be passed with a grade >= 5.5. The final grade is the weighted average of the components' grades. The report will be evaluated by the supervisor. For grading, there is a special evaluation form on Canvas community of the MSc Computer Science.

Literature

To be distributed individually, by the project advisors.

Additional Information Target Audience

Master Computer Science.

Custom Course Registration

For being accepted, students must contact their prospective advisor directly.

Recommended background knowledge

The student should have completed at least 42 credits of their Master programme, so that there is sufficient study content to be applied in the project.

Secure and Trustworthy Machine Learning

Course Code	XM_0179

Credits	6
Period	P3
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. ir. M.E. van Dijk
Examiner	prof. dr. ir. M.E. van Dijk
Teaching Staff	M. Chen, prof. dr. ir. M.E. van Dijk
Teaching method(s)	Study Group, Lecture

Course Objective

We will discuss various topics in the intersection of security and machine learning. The main goal of this course is to have you think critically about how and to what extent security guarantees can be given for machine learning tasks. The main objective is to study trustworthy intelligence augmentation from different perspectives.

Upon completion of this course, you will:

- Have knowledge about security problems that arise in machine learning (K&U, AK&U, C)
- Understand (mathematical) technique(s) towards remediating security problems (K&U, AK&U, C)
- Have knowledge of example(s) about machine learning for security (K&U)
- Have acquired critical thinking around machine learning and security (MJ, LS)

The acronyms above refer to the Dublin Descriptors covered by each learning objective, specifically: Knowledge and Understanding (K&U), Applied Knowledge and Understanding (AK&U), Making Judgment (MJ), Learning Skills (LS), and Communication (C).

Course Content

The course touches on various topics in machine learning and security. Examples of topics that can be included are: adversarial machine learning for image classifiers; Differential Privacy for distributed Stochastic Gradient Descent; secure multi-party computation for federated learning; the use of ML in security such as intrusion detection, analysis of silicon Physical Unclonable Function designs, GAN models generating synthetic data such that collected personal data can be discarded, and the moral character of cryptography, security, and artificial intelligence.

Additional Information Teaching Methods

Lectures (105 minutes 2x each week for 4 weeks); Group Q&A (135 minutes 1x each week for 3 weeks) during which (a) we cover lecture material for which we had no time during lectures, (b) upon student request re-explain topics and/or discuss upcoming assignments and (c) answer student questions about the lectured material and problems.

Method of Assessment

(1) At the start of the course you will write a short (but at least 1/3 page, single column) essay on your study strategy and your grade expectation:

You cannot pass the course if you don't submit by the deadline. And those who don't simply get NS "No Show" as the final grade.

(2) Questions and problems will be posted for each lectured topic. At the end of the course there will be a final in a controlled environment where a subset of the posted questions and problems for each topic will be asked. You will be able to prepare for this final by collecting and learning answers throughout the course. The final tests whether you have acquired a sufficient overall understanding. The final has a pass/fail grade.

If you do not pass the final, then you fail the course. If you pass the final, then your final grade is determined by your individually graded assignments as described below.

There is no re-sit opportunity for the final. A grade for the final from a previous year does not transfer to this year.

(3) There will be six (A = 6) individually graded assignments in the form of research surveys/essays together with your own self-evaluations of your submitted works. Each assignment is assessed based on the quality, correctness, and completeness of the solution(s) or written exposition. Each assignment is graded as a Fail (F), Pass (P), or Excellent (E).

(An F is also given for an assignment if you do not meet its deadline for submission.)

The following explains in text a rather complicated and intuitive formula for computing the final grade. A visual picture will be presented at the start of the course (the editor does not allow it to be inserted here).

We set

A = # Assignments

and we compute

N = 2*(#P) + 3*(#E),

where #P indicates the number of Passes you have received and #E the number of Excellent. (Notice that A = #P + #E + #F.)

If you have submitted your study strategy essay by the deadline and if you passed the final, then your final grade is equal to

14/6 + (17/6)*(N/A) rounded to the nearest integer in {1,2,3,...,10}.

The specifics of this grading policy are subject to change if the need arises.

There are no re-sit opportunities for any of the assignments/essays. Grades for assignments from previous years do not transfer to this year.

(4) At any moment during the course the instructor can request a student to explain one or more of his/her assignments in order for the instructor to verify whether the student understands the material presented in the assignments; if the student is not able to explain his/her understanding in an assignment, then the grade for the assignment will be downgraded to a fail.

Entry Requirements

The course attempts to be self-contained and explains the necessary machine learning concepts as well as security and crypto techniques. Nevertheless, students are required to be independent and study extra where their mathematical background is limited. For example, you will want to know how to compute gradients, know about the structure of a proof, can read mathematical notation, etc.

Literature

Slide decks, papers, and reference material where needed.

Additional Information Target Audience

Master Computer Security; Master Computer Science; Master Artificial Intelligence

Secure Computation

Course Code	XMU_0065
Credits	6
Period	P5
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	J.V.A. Tielen MA
Examiner	
Teaching Staff	
Teaching method(s)	

Course Objective

This is an UvA course. Please follow the link for the course description.

Course Content

https://studiegids.uva.nl/xmlpages/page/2025-2026/zoek-vak/vak/130407

Additional Information Teaching Methods

-

Method of Assessment

-

Custom Course Registration

This course is offered by the UvA. For more information contactFNWI Education Service Centre.

Enrolment via https://glass.uva.nl is required.

Security Experiments and Measurements

Course Code	XM_0098
Credits	6
Period	P3
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	prof. dr. ir. F. Massacci
Examiner	prof. dr. ir. F. Massacci
Teaching Staff	mr. S. Zhang MSc, prof. dr. ir. F. Massacci, W.B. Mbaka, dr. M. Zhang, S.A.M. van Gerwen, mr. E. Mezzi
Teaching method(s)	Seminar, Written partial exam, Lecture

Course Objective

“We don’t want high-level executives discussing whether a blue background or a yellow background will lead to more ad clicks. Why debate this point, since we can simply run an experiment to find out?” Hal Varian, Google Chief Economist [1]

The objective of this research design course is to introduce students to the critical issues behind the practical design, execution, and measurements of (security) experiments.

During the course students will

1. learn the ***scientific experimental methodology*** to determine the actual success of a security technology when applied by humans or in case control studies
2. learn some ***advanced security technologies*** (e.g. identification of vulnerability in codebases by using AI, decision making in threat intelligence; modelling threat in information systems, and one additional topic such as security in the cloud);
3. ***apply the technology*** some concrete studies as experimental subjects;
4. ***analyze the results of the experimental studies*** as security researchers to determine if such technology actually works.

The course is organised along the principles of ***scientific peer reviews*** (double blind) and expects motivated students to reach the ***highest educational objective*** in Bloom’s cognitive taxonomy (***Judgement***).

Course Content

Regular and active participation in the teaching activities offered by the course (lectures, experiments and in independent study activities) will enable students to:

- understand a specific security problem and try to address it in a laboratory case study ***[Applying knowledge and understanding]***
- understand the fundamentals of the design of (security) experiments involving both humans and code ***[Applying knowledge and understanding]***
- determine measures for the impact of a treatment and evaluating its success ***[Applying knowledge and understanding, Making judgements]***
- identify and review pros and cons on the concrete problem instance by reviewing the work of their peers ***[Making judgements, Communication, Lifelong learning skills]***
- defend in public the design of an experimental security analysis ***[Applying knowledge and understanding, Lifelong learning skills, Communication]***

Additional Information Teaching Methods

The course is organised into weekly lectures coupled with practical assignments where the students will apply gained theoretical knowledge on one or more case studies of a software system.

- ***Lectures and discussion sessions*** will introduce the students to four security technologies, show how they can be applied to some scenarios of moderate complexity, and provide additional material to study.
- ***Experiments in class in presence.*** Each student will be asked to apply the security technologies on which s/he has received training on some concrete scenarios of moderate complexity. Students should demonstrate that they are able to understand the scenarios.
- ***Scientific Reports.*** Each student will write and submit some scientific reports to analyse the results of the experiments.
- The ***first version of the reports*** will include an analysis of the performed experiments as in scientific articles
- The ***revised report(s)*** include a rebuttal of the received reviews on the previous version as in scientific articles
- Students who do not submit by the deadline will not be able to review the reports of their peers.
- ***Individual peer reviews:*** each student will review the assignments submitted by their peers according to a detailed grading scheme provided and exemplified by the lecturers
- Each student will have to review five assignments for each type of submission.
- The feedback of students of past editions of the course is that the average time for peer evaluation is estimated at ***one hour per (good) review.***
- ***Class presentation and public review.*** To provide intermediate feedback and to learn to identify and review a variety of applications of the technique students will be asked to discuss their assignments in front of the class.
- Attending the discussion classes is ***mandatory***, also for students that have already discussed in previous sessions.
- **Q&A sessions** (online) will be organized to answer questions.

The final list of experiments will be determined before the course starts. The following experiment happened in the previous years:

- ***Modelling Threats:*** students playing the role of analysts will have to decide whether it is easier to identify a risk by using a graphical model or by receiving textual information from a Large Language Model;
- ***Finding vulnerabilities in code:*** students playing the role of developers will try to identify vulnerabilities in a large file without support or when given a subset of the lines of code as selected by a ML/Static analysis algorithm;
- ***Threat Intelligence:*** students playing the role of intelligence analysts will have to decide whether the intelligence received by an AI algorithm is more credible than the one received by a human expert;
- Make your own: an experiment proposed by the students themselves.

Method of Assessment

Given the tight timeline, at the ***beginning of the course*** students will have to submit a short (1/3 page) essay on their study strategy and grade expectation and participate in a simple online experiment on threat assessment. Those that do not submit by the deadline will get a NS "No Show" as the final grade.

The ***overall grade*** is determined as follows:

70% for the ***Research Reports*** each of which is divided as

- 65% on the evaluation of the report as graded by the peers
- 35% for the participation in the peer review process and the quality of the reviews as re-evaluated by the lecturers and teaching assistants

25% for the ***Active Participation in Experiments***

5% for the design of a "Make Your Own"***Experiment Proposal***

Submission of Intermediate Results

- Each report is submitted weekly by a deadline. If a student does not submit a report this specific report will be counted a zero point
- If a student does not submit his/her own report, it cannot do the reviews of the other students reports
- Reviews are also submitted weekly by a deadline, every review that is not submitted by the deadline will be counted zero points.
- If a student submit some of the reports, or participates to some of the experiments and cannot reach 5.5 it will be marked NVD
- The only way to obtain a NS is not to participate to any activity.

Pass or fail

- ***Quality of the reviews.*** Additional points beyond the participation will be deducted for students who write poor (e.g. generic such as "it is ok, full points") or unfair reviews.
- ***Presentation of the reports in class***. If a student is asked to present his/her report in class and is absent , do not respond, or not able to do present it ("I do not remember what is in the report I submitted yesterday"), the report will be considered void.

Plagiarism and Fraud Check

If the number of students is so large that discussions in front of the class cannot be organised for all students to present at least once, a digital exam will be organised to assess the knowledge of the reviewed material. Students who have not presented must score ***at least 6/10*** in the confirmation exam for the assignments score to be considered valid.

Resit

It is not possible to resit the assignments and the peer reviews. Students can only resit failed reviews due to additionally deducted points by writing a report analyzing the reviews of all other students for each failed report.

Carry on of previous results from previous years

- This is generally not possible as the reports are delivered in installments and each report must address in the subsequent installment the peer review comments of the students of the course with the exception below:
- If a report and its peer review process and revision are entirely completed (Report submitted, peeer review done, report revised, peer review done) then it is possible to carry on the result provided the report has not being voided due to plagiarism/failure of discussion
- If the experiment is the same, students can recycle and adjust the same text and scripts of their own report from the previous year by simply updating the results with the new data.
- it is only possible to carry on the participation to the experiments if one has participated to all experiments the previous year.

Entry Requirements

See recommended knowledge.

Literature

Assumed pre-requisite

- Alan Agresti, Christine A. Franklin, Bernhard Klingenberg. Statistics: The Art and Science of Learning from Data, 4th Edition. Pearson Education.
  This is a classic textbook of statistics, typically used in BSc courses. Chapters 4 and 5 contain a useful summary that defines experiments, case-control studies, and prospective studies. Any other statistics text will work.

Necessary Material

- Lectures on security topics material, training for the experiments etc. will be available on CANVAS.

Useful

- Andrew Jaquith . Security Metrics: Replacing Fear, Uncertainty, and Doubt. Wiley.
  This is a classic book on measuring security. It is almost 13 years old and written a bit as a set of anecdotes. Still, it covers essentially all metrics you'll find in the industry and a rough description of all processes. We will use in particular the chapter on visualizing security metrics
- Rachel Glennerster and Kudzai Takavarasha. Running Randomized Evaluations: A Practical Guide. Princeton University Press.
  This is not a book about computer security. It is a book about social development and evaluating the impact of "interventions" (aka experiments) on poverty eradication. However, if one does a "search-and-replace" of the words "deworming program" with "vulnerabilities finding program" and the word "children" with "developers" it provides a good discussion of all problems that a security and computer science experimenter faces and how to solve them.
- Judea Pearl, Madelyn Glymour, Nicholas P. Jewell. Causal Inference in Statistics - A Primer. Wiley
  A thin booklet on the notion of causality, and how we can formally capture it. We will use causal graphs to visualize the concepts behind our case studies. We will use the first two chapters.
- Cited [1] Michael Luca and Max H. Bazerman "Want to Make Better Decisions? Start Experimenting"MIT Sloan Management Review June 04, 2020. Interview with Hal Varian.

Additional Information Target Audience

Master Computer Science
Master Computer Security
Master Artificial Intelligence

Additional Information

"***Participating as a subject***" means that each individual student will read or watch a training specific to the task (e.g. how to identify a Java vulnerability on source code), then will be assigned a task (e.g. find vulnerabilities in the following two random files, one filtered by a Machine Learning tool a second just the raw code from a file), and execute the task to the best of one's ability within a time limit (e.g. report the vulnerable lines within 2 hours).

"***Analyzing the results***" means that each student will receive all data collected in the previous steps (plus some other additional data) and will have to assemble a report describing the experiment so that other people could replicate it, the results of the experiments in their opinion (e.g. plots showing that there was no difference between the file processed by the ML and the unaided human, statistical analysis, etc.) and what the criticality and threats to validity of the experiments were (now directly from their own experiments).

All reviewers are anonymous (to the peers) but not to the teachers who act as program chairs in a scientific conference.

Custom Course Registration

Given the tight timeline, at the beginning of the course students will have to submit a short (1/3 page) essay on their study strategy and grade expectation and participate in a simple online experiment on threat assessment. Those that do not submit by the deadline will get a NS "No Show" as the final grade.

Explanation Canvas

The submission of the report and the peer review of the reports will be done in Canvas/FeedbackFruit.

Because of the way the system works by automatically assigning reviewers, it is not possible to grant any deadline extension for the submission of the reports and the submission of the reviews.

Students should therefore carefully plan their activities and submit also a draft version of their reports well in advance of the deadline (multiple submissions are possible) rather than waiting for the last moment.

Recommended background knowledge

Basic general knowledge of statistics at BSc level are assumed (mean, standard deviation, concept of distribution and confidence interval, and simple statistical tests). A basic data analysis code will be supplied by the teacher.

Good working knowledge to write scripts or programs to manage data and software repositories is useful to avoid manual analysis. Scripts for analyzing data can use Matlab, R or Python. Use of such languages is up to the students.

Good analytical knowledge of programming languages (e.g. Java, C, Python,..) and software testing is key to being able to perform the analysis of code vulnerabilities. Introductory knowledge of software security is also assumed.

Service Oriented Design

Course Code	X_405061
Credits	6
Period	P1
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. J.R.E. Bogner MSc
Examiner	dr. J.R.E. Bogner MSc
Teaching Staff	prof. dr. P. Lago, dr. J.R.E. Bogner MSc
Teaching method(s)	Seminar, Lecture

Course Objective

Upon completion of this course, students will be able to:

- Analyze a design problem and model software for it in terms of independent services by using notations from industrial practice (Knowledge and Understanding; Applying Knowledge and Understanding; Making Judgments; Communication; Learning Skills).
- Apply structured techniques to make and justify design decisions for service-oriented software (Knowledge and Understanding; Applying Knowledge and Understanding; Making Judgments; Communication; Learning Skills).
- Navigate quality and sustainability requirements and understand how to achieve them in service-oriented design (Knowledge and Understanding; Applying Knowledge and Understanding; Making Judgments; Communication; Learning Skills).
- Collaborate in a team and synchronize their project contributions into one holistic presentation (Applying Knowledge and Understanding; Making Judgments; Communication; Learning Skills).

Course Content

Throughout the course, students will work in teams to provide a solution for a case presented by invited guests from an external company or organization (case providers). Students will analyze the problem domain and design a service-oriented solution for it.

For this project, the lectures explain the required theoretical concepts about service orientation as a software paradigm, as well as the difference between service-oriented architecture and microservices. More importantly, the lectures introduce techniques for identifying the requirements for service-oriented software, how to map them to business services, and how to transform them into complex networks of software services. Several UML-based modeling techniques and diagrams are used in the course. Special emphasis is given to design reasoning techniques for decision-making, service identification, service-oriented software design, and sustainability concerns. Lastly, the course also touches on RESTful API design and service-oriented design patterns.

Additional Information Teaching Methods

The course consists of three live sessions per week (each 01:45 h long):

- Lectures: acquire necessary theory and understand used modeling techniques
- Project sessions: discuss your team progress with your TA and other teams (mandatory attendance)
- Presentation sessions: present your project to all other teams in a plenary session

Method of Assessment

The grade is mostly determined by two team-based and project-based assignments (A1 and A2). A1 focuses on the selection and analysis of the design space for the chosen project. A2 revises and extends A1 and proposes a service-oriented design solution.

Individual contribution to the project:
Each assignment has parts that count for the whole group and parts that count only for an individual student. In general, the project workload should be distributed fairly among the group members. Insufficient contribution to the group parts of the assignment can lead to failing the course. The final grade is the average of both assignment scores, plus potential bonuses (see below).

Project presentation:
In addition to the individual sections per assignment, each student is expected to both attend and contribute actively during the weekly **project sessions** with the TA. Attendance is documented, and missing too frequently will lead to failing the course. Moreover, each student needs to fulfill two roles in the controlled environments of these sessions at least once over the course of the project.

- **Presenter:** present the progress of your team and answer questions
- **Discussant:** moderate the discussion following another team's presentation and ask some questions yourself

Each of these roles will be assessed with either *fail*, *pass*, or *pass and +0.15*.
Lastly, each student also needs to have an active part in the **plenary presentation session** when their team is presenting to everyone. This part is also assessed with *fail, pass*, or *pass and +0.15*.
The three potential bonuses (up to a maximum of +0.45) will be applied to the average of the two assignment scores. Receiving a *fail* in any of these three roles will lead to failing the course.

Canvas quizes in lectures:
In each lecture, a Canvas quiz takes place that can provide a bonus to the final grade (+0.04 per quiz, for a total of +0.28). These Canvas quizzes do not impact the grade negatively.

Final grade:
To pass the course, a student must meet the following criteria:

- Contributed adequately to the group parts of A1 and A2
- Did not miss more than 2 project sessions with the TA

- Did not receive a *fail* for the project presenter role
- Did not receive a *fail* for the project discussant role
- Did not receive a *fail* for presenting in the plenary presentation session
- Both A1 and A2 have a score of at least 6.0

If these criteria are fulfilled, the final grade is calculated as follows:
(A1 score + A2 score) / 2 + project presenter bonus + project discussant bonus + plenary presenter bonus + Canvas quiz bonus
This number is then rounded to the nearest 0.5, e.g., an 8.67 is rounded to 8.5, while an 8.81 is rounded to 9.0.

Mode of re-examination:
Each failed project assignment (A1 and A2) can be revised and resubmitted once (*resit*). Failing the other mandatory contributions can be recovered in the following ways:

- Missed more than 2 project sessions with the TA without reasonable explanation, e.g., medical issues: take over additional shared project-tasks from your team members to make up for your missed attendance
- Received a *fail* for the presenter role: privately presenting the project to the lecturers
- Received a *fail* for the discussant role: acting as discussant in one of the plenary presentation sessions or privately discussing your project with the lecturers
- Received a *fail* for presenting in the plenary presentation session: taking over the presenter role for your team more often in the project sessions or privately presenting the project to the lecturers

Literature

All course material will be distributed online on Canvas.

Additional Information Target Audience

MSc Computer Science
MSc Computer Security

Custom Course Registration

Further information for this course will be made available on Canvas, in which all students must be enrolled. Group enrollment also takes place via Canvas and a shared spreadsheet.

Explanation Canvas

Instructions about group enrollment and the course process will be provided at least one week before the course starts. All students must be enrolled before then.

Recommended background knowledge

- At least intermediary software design and modeling knowledge
- At least intermediary knowledge of the Unified Modeling Language (UML)
- At least intermediary programming skills, even though no code will be written in the projects

Software Language Engineering

Course Code	XM_0172
Credits	6
Period	P4
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. M. Verano Merino MSc
Examiner	dr. M. Verano Merino MSc
Teaching Staff	dr. M. Verano Merino MSc
Teaching method(s)	Seminar, Lecture

Course Objective

Software development is a very dynamic activity. Language-oriented programming (LOP) is a programming paradigm that focuses on designing and implementing domain-specific languages (DSLs) to solve problems within specific domains (e.g., SQL, HTML). Instead of using general-purpose programming languages, LOP emphasizes

the creation of software languages tailored to particular needs. This approach can improve productivity and reduce errors by making the programming process more intuitive and less error-prone.

In this course, students will:

- explore software development through a LOP approach; (Knowledge and Understanding) (Apply knowledge and understanding)
- learn to explore related literature; (Knowledge and Understanding)
- learn to present related research papers; (Communication) (Learning skills)
- learn to design and implement DSL artifacts using specialized language technology (e.g., language workbenches). (Apply knowledge and understanding) (Making Judgements) (Learning skills)

Course Content

In this seminar course we will unveil the foundations of software language engineering by diving into the details of the design, implementation, and evolution of software languages. Students will be developing their own DSL and their tooling. Upon completion of the course, students will get an understanding of most of the following topics:

- Implement a fully-fledged software language and its tooling (e.g., IDE). How to conduct a domain-analysis.
- How to design and implement the concrete and abstract syntax of a software language using grammarware and modelware technologies.
- How to design and implement the semantics of software languages.
- How to use and create tools for a software language.
- Type checkers
- Editors
- IDE services (e.g., syntax highlighting, auto-completion, folding)
- Use parser generator technologies.
- Understand and apply model-to-model transformations.
- Understand and define code-generators to produce executable code.
- Usage of different language workbench technologies (e.g., Rascal, Eclipse EMF, and Jetbrains MPS).

Additional Information Teaching Methods

The lecture will mostly be in a seminar format. Were we will discuss language engineering research papers and tools. Students will read research papers before the lecture, write short reports on the material, prepare presentations and discuss the main points during the lecture with the rest of the class.

Also, there will be practical assignments where students implement language engineering artifacts (e.g., syntax of a language, type-checkers, code generators) related to the week's topic.

Method of Assessment

The course is focused on analyzing existing research on Software Language Engineering, and applying those concepts to practical examples. Therefore, the course contains several programming assignments to assess the core topics of the course. Finally, it is expected that at the end of the course, students will develop a project in pairs where they implement their own DSL.

The grading distribution is as follows:

- Final Project (70%, group) consists of the implementation of a fully functionalDomain-Specific Language. This grade is based on the following components:

- Implementation: fully functional Domain-Specific Language.

- Coding interview: where each student individually demonstrates understanding of their group's implementation and design choices.

- Reflection journals (pass/fail): documenting weekly contributions and learning progress of each team member.

- The Scientific Report (20%, group) is an academic report documenting your DSL, including domain analysis, design rationale, implementation details, and evaluation.
- The Seminar Presentation (10%, group) involves presenting an assigned research paper in the field of Software Language Engineering.

To obtain a passing grade for the course, you must pass the reflection journals. The grade for the Final Project and the Scientific Report must each be at least 5.5 to pass the course. There is no resit opportunity for any component.

Literature

Mandatory Literature:

- Lämmel Ralf. (2018). Software languages : syntax, semantics, and metaprogramming. Springer. https://doi.org/10.1007/978-3-319-90800-7 [Available online]
- Andrzej Wąsowski , Thorsten Berger (2023). Domain-Specific Languages Effective Modeling, Automation, and Reuse. https://doi.org/10.1007/978-3-031-23669-3 [Available online]

Recommended Literature:

- Nystrom, Robert. Crafting interpreters. Genever Benning, 2021. [Available online]
- Brambilla, M., Cabot, J., Wimmer, M. (2017). Introduction. In: Model-Driven Software Engineering in Practice. Synthesis Lectures on Software Engineering. Springer, Cham. https://doi.org/10.1007/978-3-031-02549-5_1 [Available online]
- Marjan Mernik, Jan Heering, and Anthony M. Sloane. 2005. When and how to develop domain-specific languages. ACM Comput. Surv. 37, 4 (December 2005), 316–344. https://doi.org/10.1145/1118890.1118892 [Available online]
- Arie van Deursen, Paul Klint, and Joost Visser. 2000. Domain-specific languages: an annotated bibliography. SIGPLAN Not. 35, 6 (June 2000), 26–36. https://doi.org/10.1145/352029.352035 [Available online]
- Sebastian Erdweg et al. Evaluating and comparing language workbenches: Existing results and benchmarks for the future, Computer Languages, Systems & Structures. Volume 44, Part A, 2015, Pages 24-47, https://doi.org/10.1016/j.cl.2015.08.007. [Available online]

Additional Information Target Audience

This course is targeted at students interested in understanding DSL design and implementation and engineering programming-related tools.

Recommended background knowledge

Students must have a solid programming background.

Software Security

Course Code	X_400127
Credits	6
Period	P1
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. E. van der Kouwe
Examiner	dr. E. van der Kouwe
Teaching Staff	prof. dr. ir. H.J. Bos, dr. E. van der Kouwe
Teaching method(s)	Lecture

Course Objective

At the end of the course students will:

- have knowledge of the concept of vulnerability in design and system implementation of applications. (K&U)
- (by doing) deeply understand the concepts of memory corruption attacks (such exemplified by buffer overflows, use-after-free, etc), (K&U, AK&U, MJ)
- (by doing) deeply understand mitigations (such as control flow integrity, address space layout randomization, and shadow stacks) as well as some of their weaknesses (K&U, AK&U, MJ)
- have hand-on experience with basic and advanced exploitation and mitigation methods, including state-of-the-art research in systems security (K&U, AK&U, MJ)

The acronyms above refer to the Dublin Descriptors covered by each learning objective, specifically: Knowledge and Understanding (K&U), Applied Knowledge and Understanding (AK&U), Making Judgment (MJ), Learning Skills (LS), and Communication (C).

Course Content

This is a course on security with a focus on systems work. The course covers a wide spectrum of security issues.

We explicitly focus on systems security rather than cryptography, as we want to show students how attackers penetrate systems and most security issues are not due to bad crypto.

Specifically, the course focuses on memory corruption and application security (buffer overflows, format string bugs, dangling pointers, shellcode, return-oriented programming, ASLR/DEP/canaries, control flow integrity), and state-of-the-art attacks and defenses (latest research insights).

Much of the course will be hands-on and challenge-based. In assignments, students will carry out and investigate attacks in a controlled environment. This involves programming at the lowest level (in assembly).

Additional Information Teaching Methods

Lectures and (very challenging) practical assignments.

Method of Assessment

Written exam (30%) and practical assignments (70%). The exam must be passed with a grade >= 5.5. Each of the practical assignments must be passed with a grade >= 4.0. The final assignment grade is the weighted average of the assignments' grades. Each assignment must be completed individually.

Please note that your grade will be registered as follows:

- For the first exam date: if you did not submit any assignments and did not participate in the exam, the result will be registered as no show (NS).
- For the second exam date: if you did not participate in the resit exam, the result will be registered as no show (NS).
- In all other cases, you will receive either a numeric grade or incomplete (NVD).
- If you did not meet the minimum grade requirements for the assignments and/or the exam, but your weighted average is >= 5.5, your grade is NVD.
- Otherwise, your final grade is your weighted average, rounded to half points except for 5.5.
- If you plan to drop the course, do not submit anything, as NVD is administratively considered a failing grade.

There is no resit opportunity for the practical assignments. Results for practical assignments and for the exam may be transferred from previous years if their contents did not change significantly since the result was obtained (as determined by the examiner). If you submit an assignment or participate in the exam, the new result replaces any transferred results.

We ensure that each student individually demonstrates their abilities through the requirement that the supervised exam be passed with at least 5.5 and through active plagiarism detection for the assignments.

Literature

No set book. All material will be made available during the course.

Additional Information Target Audience

Master Computer Security
Master Computer Science

Recommended background knowledge

No formal requirements, except a keen interest and sufficient time.
Programming experience in C is essential. Knowledge of assembly and computer architecture helps too.

Software Testing

Course Code	X_400439
Credits	6
Period	P5
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. N. Silvis-Cividjian
Examiner	dr. N. Silvis-Cividjian
Teaching Staff	dr. N. Silvis-Cividjian

Teaching method(s)	Practical, Written partial exam, Lecture

Course Objective

After completing this course, students will be able to:

1. Appropriately use terms specific to the testing process. (Knowledge and understanding) (Applying knowledge and understanding)
2. Analyze a software-related failure accident and report on it. (Applying knowledge and understanding) (Making judgements) (Communication)
3. Given a software requirements specification (SRS) and an existing software product that can be standalone or embedded in a device, design and implement a comprehensive test strategy and produce testing documentation according to modern software engineering standards. (Applying knowledge and understanding) (Making judgements) (Communication) (Lifelong learning skills)
4. Practice software testing in a professional, responsible, and ethical manner. (Lifelong learning skills) (Making judgements)
5. Identify possible discrepancies between software testing as taught in an academic context and software testing as it happens in real IT companies. (Knowledge and understanding) (Lifelong learning skills)

Course Content

Testing is a method to improve software quality. Realistically, software testing is a trade-off between budget, time, and quality. It is impossible to test everything, so choices have to be made. Students learn how to make and justify these choices and systematically test a software product (standalone or embedded in a smart system).

This course teaches students how to test software in the broad context of systems engineering. It provides an introduction to software testing with an emphasis on technical activities like test generation, selection, execution, and assessment, and on non-functional requirements such as safety and security.

The course tries to answer a few questions, such as:

- How to learn from accidents?
- How to design adequate test cases in a limited time?
- When to stop testing?
- How to test a safety-critical software-intensive system?
- What are the challenges of embedded software testing?

A few guest lectures showing examples of testing in the industry area also planned.

Topics: boundary value analysis, equivalence partitioning, combinatorial testing, model-based testing, control-flow testing, data-flow testing, mutation testing, regression testing, inspections, automated testing, integration testing, system testing, hazard analysis, fuzz testing, diagnostic tools for embedded systems testing.

Additional Information Teaching Methods

Lectures and mandatory practical.

Method of Assessment

The GROUP work is graded with a pass/fail and is evaluated through a few face-to-face interviews with the teaching team. The Individual component has two parts, both proctored: a bug hunting exam in a tool called DBugIT and an exam in Testvision. A pass for the GROUP component and a grade of >= 5.5 for both exam components are needed to pass the course.

Literature

- A. Mathur, Foundations of software testing, Addison-Wesley Professional; 2nd edition (February 13, 2014), 2014, *ISBN: * 978-8131794760.
- Lecture notes, and author's textbook (in press).

Additional Information Target Audience

MSc Computer Science
MSc Computer Security
MSc Computational Science

Explanation Canvas

All course materials are available on Canvas.

Recommended background knowledge

Basic knowledge of programming (Python, C++) and software engineering (requirements engineering) is required.

Systems Seminar

Course Code	XM_0122
Credits	6
Period	P4
Course Level	600
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	B. Chandrasekaran
Examiner	
Teaching Staff	dr. D. Bonetta, dr. T. De Matteis, B. Chandrasekaran
Teaching method(s)	Seminar, Practical, Lecture

Course Objective

The key objective of this seminar is to introduce students to designs, details, and challenges associated with building modern computer systems (CompSys) from a single machine to distributed services. As part of this exercise, students will read, review, present, and discuss recently published cutting-edge research on the design and implementation of computer and networked systems.

The course takes the form of an interactive seminar (presentations and discussions) mostly driven by the students. Upon completion of this course, students will be able to:

- Reason about various design choices, systems building trade-offs, and ongoing and emerging research challenges for a design based on the scale, deployment goals, and capabilities provided. [Applying knowledge and understanding] [Making judgements]
- Understand recent advancements in systems (leveraging modern hardware - SmartNICs, fast storage, FGPAs, programmable switches) and cutting-edge techniques to run these systems safely and efficiently. [Knowledge and understanding]
- Read papers, write critical (technical) reviews (style, tone, details), compare details across other papers, understand how to evaluate a paper, and develop and learn about research methodology skills. [Knowledge and understanding] [Applying knowledge and understanding] [Communication]
- Learn how to present and lead discussions on state-of-the-art literature in an interactive setting with your peers [Communication] [Lifelong learning skills]

Course Content

The course is largely student-driven. Each week we will have paper presentations and discussion sessions where we will discuss recently published, high-impact papers in the literature. We will provide a curated list of recently published papers in the first week of the class to cover these topics. The acquisition of this knowledge is important to understand how the fields of computer systems are emerging, and how future cloud/distributed systems in practice will be shaped.

In each class session there will be a presentation(s) from students, and students will lead the discussion on the topic.

Outside the class, the students need to read papers and write detailed technical reviews.

Additional Information Teaching Methods

In each lecture session we will discuss a few closely related papers in detail. Every student is expected to read all papers and send weekly reviews before the week starts. A sample format for the review and previous year's review samples will be provided.

There will be no deadline extensions and we will not allow late submissions.

In each lecture there will be presentations (15-20mins) from students, and then an open, conference program committee (PC) style discussion led by students and teachers regarding the (i) content and contributions; (ii) strengths; and (iii) weaknesses (and what could be improved); (iv) how papers which are discussed are related to

each other; (v) various other issues raised in your submitted reviews; and (vi) overall broader research direction to which the papers belong. We expect the students to actively participate in these presentations.

Method of Assessment

Students are awarded points every week on the quality of the review (technical correctness, understanding of the paper, details covered, tone and style), and the presentations (style, delivery, visuals, and audience engagement). The final grade is a weighted average of points: Reviews (60%) and presentations (40%).

Reviews: The papers reviewed will be from one or more distinct research areas covered in the course. You must score at least 5.5 in the reviews (in total) and 5.5 on average in the reviews belonging to each of the different research areas.

Presentation: Each student must present a paper, failing which you will not be able to pass the course; there is no minimum score or cutoff for the presentation though.

- The final grade is calculated as a weighted sum of points obtained for the reviews and presentation, as shown above.
- There are no resits available for this course.
- This course does not allow students to carry over (partial) grades from last year.

[Fraud] The course is made fraud resistant by having selective and specialized reviews to assess the knowledge of the students and deep technical presentations, both of which are graded individually.

Please note that your grade will be registered as follows:

If you do not submit any review and do not present a paper, you will receive a NS.
In all other cases, you will receive either a numeric grade or incomplete (NVD).
If you did not meet the minimum grade requirements for the reviews, your grade will be NVD.
Otherwise, your final grade is your weighted average.
If you plan to drop the course, do not submit anything, as NVD is administratively considered a failing grade.

Literature

A list of recently published and classic papers (mandatory and background readings) reflecting recent developments.

A sample format for the review and previous year's review samples will be provided. We will also introduce advanced topics, reproducibility research methodology, and tips for reviewing and presenting works.

Additional Information Target Audience

MSc Computer Security
MSc Computer Science

Recommended background knowledge

This course builds on a wide variety of computer systems and networking knowledge and concepts such as design of operating systems, networking and storage stacks, integration of accelerators, memory virtualization and management, network architecture, Internet-scale computing, process management, failure and fault tolerance in distributed systems, computer architecture (interrupts, CPU caches, multi-core systems, I/O buses), etc. Due to the nature of the course, there will be limited opportunity to learn these concepts in the class. In case you lack necessary background knowledge, ask us early to provide background reading material that you should cover by yourself.

Term Rewriting Systems

Course Code	XM_400121
Credits	6
Period	P4
Course Level	500
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. J. Endrullis
Examiner	dr. J. Endrullis
Teaching Staff	dr. J. Endrullis

Teaching method(s)	Seminar, Lecture

Course Objective

The students will learn to:

- reason about properties of abstract reduction systems, (Applying knowledge and understanding) (Making judgements)
- understand term rewriting and be able to design term rewriting systems, (Knowledge and understanding) (Applying knowledge and understanding)
- understand and apply combinatory logic, (Knowledge and understanding) (Applying knowledge and understanding)
- reason about and prove termination of TRSs (using recursive path orders, monotone algebras,...), (Applying knowledge and understanding) (Making judgements)
- reason about confluence of TRSs (using critical pairs and orthogonality), (Applying knowledge and understanding) (Making judgements)
- apply Knuth-Bendix completion, (Applying knowledge and understanding)
- understand and apply different reduction strategies, (Knowledge and understanding) (Applying knowledge and understanding)
- reason about properties of TRSs using modularity, (Applying knowledge and understanding) (Making judgements)
- understand (un)decidability of important properties, (Knowledge and understanding)
- understand the basic concepts of infinitary rewriting. (Knowledge and understanding)

Course Content

Term rewriting systems (TRSs) provide a natural formalism for specifying rules of computation and investigating their properties. TRSs are also of basic importance for functional programming and for the implementation of abstract data types. Applications can also be found in theorem proving, proof checking, and logic programming. During this course, students will learn the fundamental notions of term rewriting and getting acquainted with some more advanced topics in the field.

Additional Information Teaching Methods

Lectures and exercise classes.

Method of Assessment

At the end of the course, there is a written final exam. The final grade is the grade of the final exam.

Literature

Course notes will be provided on Canvas.

Additional Information Target Audience

MSc Computer Science

The Social Web

Course Code	X_405086
Credits	6
Period	P2
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. D. Ceolin
Examiner	dr. D. Ceolin
Teaching Staff	dr. F. Ilievski, dr. D. Ceolin, Z. Inayat
Teaching method(s)	Computer lab, Lecture

Course Objective

In this course, students will:

- Learn theory and methods concerning communication and interaction in a Web context (Knowledge and Understanding).
- Understand how user data is represented and distributed across devices in the context of the Social Web (Knowledge and Understanding).
- Collect and prepare Web data, apply the above skills in practical assignments (Apply Knowledge and Understanding)
- Reflect on social and ethical responsibilities linked to the application of these methods (Making Judgements).
- Communicate knowledge and judgements in a final paper (Communication).

Course Content

This course will cover theory, methods and techniques for:

- personalization for Web applications;
- Web user & context modeling;
- user-generated content and metadata;
- multi-device interaction;
- usage of social-web data.

The course takes a data science approach, using Jupyter notebooks to collect and analyse data, with a research paper describing the data science study as the main outcome of the course.

Additional Information Teaching Methods

Lectures (1x week)
Practical sessions (1x week)
Presentation seminars (1x week)

Method of Assessment

The weighted average of:

- 70% research paper (group assignment A),
- 10% group presentation (P),
- and 20% individual exam (E).

A minimum grade of 5.5 for each component is required.

Compensation: it is not possible to compensate one component with another.

Mode of re-examination: if insufficient, each component may be revised and resubmitted once.

Literature

- Course lecture slides;
- Selected articles, videos and Web links

All materials for each lecture will be provided through Canvas.

Additional Information Target Audience

Master Artificial Intelligence
Master Computer Science
Master Information Sciences
Master Computer Security

Additional Information

The course has a maximum enrolment limit of 120. The students of the Master Information Sciences, for whom it is compulsory, have priority over the students that take this course as an elective or a constrained choice course.

The students that take this course as an elective or a constrained choice course run the risk of not being able to participate, even if they have registered on time, if the maximum enrolment limit has been reached.

The students of the Master Information Sciences, for whom it is compulsory, run the risk of being initially refused if the enrolment limit has been reached at the time of registration. However this will be adjusted manually after registration has been closed so that all the students of the Master Information Sciences will be able to participate.

Recommended background knowledge

Basic Python programming skills.

Verification for Security

Course Code	XM_0099
Credits	6
Period	P1
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	dr. K. Freiherr von Gleissenthal
Examiner	dr. K. Freiherr von Gleissenthal
Teaching Staff	dr. K. Freiherr von Gleissenthal
Teaching method(s)	Seminar, Lecture

Course Objective

In this course, students get acquainted with techniques for program verification with a focus on low user effort and automation and learn how to apply these techniques to prove that programs are secure.
After completing the course, students will:

- Have learned the basics of checking correctness proofs using SMT-solvers. (Knowledge and Understanding)
- Have learned how to encode program correctness proofs in SMT via Hoare Logic. (Knowledge and Understanding, Applying Knowledge and Understanding)
- Understand and know how to apply algorithms for computing correctness proofs automatically. (Knowledge and Understanding, Applying Knowledge and Understanding)
- Have learned how to use verification techniques to prove that programs are secure, that is, they don’t leak sensitive data directly, or indirectly through the cache, and timing. (Knowledge and Understanding, Applying Knowledge and Understanding)
- Have gained practical experience in applying these techniques. (Applying Knowledge and Understanding)
- Understand the limitations of verification techniques and gain an understanding of when to apply them. (Knowledge and Understanding, Applying Knowledge and Understanding, Making Judgements)

Course Content

When is a program correct? When is it secure? It is often hard to convince yourself that a program works for a given set of inputs, let alone for all possible inputs and without inadvertently leaking confidential information. In this course, you will learn how to formally reason about---and prove---that a program is correct and secure. Since life is short and writing proofs is difficult, we will focus on low-effort techniques and try to let the computer automatically discover proofs for us. We will start with proving general program properties, say that a program never reaches a bad state, and then move on to proving security and confidentiality, more precisely, that a program does not leak secrets, directly, or indirectly via cache, timing, or micro-architectural side-channels like Spectre. We will study verification not purely in theory, but instead, we will implement verification methods and get hands-on experience applying them.

Additional Information Teaching Methods

Lectures and practical assignments.

Method of Assessment

Written exam (50%) and practical assignments (50%). The exam must be passed with a grade >= 5.5. Each of the practical assignments must be passed with a grade >= 4.0.

Please note that your grade will be registered as follows:

- If you do not submit any assignment and do not participate in the exam, you will receive a NS.
- In all other cases, you will receive either a numeric grade or incomplete (NVD).
- If you did not meet the minimum grade requirements for the assignments, your grade is NVD.

If you plan to drop the course, do not submit anything, as NVD is administratively considered a failing grade. There is no resit opportunity for the practical assignments.

Results for practical assignments and for the exam may be transferred from previous years if their contents did not change significantly since the result was obtained (as determined by the examiner).

We ensure that each student individually demonstrates their abilities through the requirement that the supervised exam be passed with at least 5.5, through active plagiarism detection for the assignments, and by investigating whether each group member contributed sufficiently to the group assignments.

Literature

No set book. All material will be made available during the course.

Additional Information Target Audience

MSc Computer Security
MSc Computer Science

Recommended background knowledge

Students are expected to know functional programming. The assignments use advanced features such as typeclasses and monads.

Web Services and Cloud-Based Systems

Course Code	XMU_418110
Credits	6
Period	P4
Course Level	400
Language Of Tuition	English
Faculty	Faculty of Science
Course Coordinator	J.V.A. Tielen MA
Examiner	
Teaching Staff	
Teaching method(s)	

Course Objective

This is an UvA course. Please follow the link for the course description.

Course Content

https://studiegids.uva.nl/xmlpages/page/2025-2026/zoek-vak/vak/130392

Additional Information Teaching Methods

-

Method of Assessment

-

Custom Course Registration

This course is offered by the UvA. For more information contactFNWI Education Service Centre.

Enrolment via https://glass.uva.nl is required.